Ayyash Zamny

25-26J-455_Thesis_Draft

Undergrad Thesis Checking

Document Details

Submission ID

trn:oid:::3618:136645180

Submission Date

Apr 26, 2026, 6:21 PM GMT+5:30

Download Date

Apr 26, 2026, 7:33 PM GMT+5:30

File Name

25-26J-455_Thesis_Draft.pdf

File Size

952.5 KB

70 Pages

15,180 Words

91,706 Characters

5% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Small Matches (less than 8 words)

Match Groups

50 Not Cited or Quoted 5%
Matches with neither in-text citation nor quotation marks

5 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

4% Internet sources
1% Publications
4% Submitted works (Student Papers)

Match Groups

- **50** Not Cited or Quoted 5%
  Matches with neither in-text citation nor quotation marks
- **5** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

4% Internet sources
1% Publications
4% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Student papers	Sri Lanka Institute of Information Technology on 2020-10-21	<1%
2	Internet	arxiv.org	<1%
3	Internet	www.coursehero.com	<1%
4	Student papers	American Intercontinental University Online on 2026-02-17	<1%
5	Student papers	Brunel University on 2026-03-27	<1%
6	Student papers	Asian Institute of Technology on 2014-05-06	<1%
7	Student papers	Ravensbourne on 2025-04-11	<1%
8	Student papers	Sri Lanka Institute of Information Technology on 2016-05-20	<1%
9	Internet	file.techscience.com	<1%
10	Student papers	Islington College,Nepal on 2026-04-19	<1%

11 Student papers

Loughborough University on 2026-04-21 <1%

12 Student papers

King's College on 2025-04-14 <1%

13 Internet

fastercapital.com <1%

14 Student papers

American University of Iraq on 2025-11-02 <1%

15 Internet

kth.diva-portal.org <1%

16 Internet

www.redbooks.ibm.com <1%

17 Student papers

Coventry University on 2026-04-13 <1%

18 Student papers

Islington College,Nepal on 2026-04-22 <1%

19 Publication

Jaskaran Singh, Meenu Gupta, Rakesh Kumar. "Smart Technologies and Intelligen... <1%

20 Student papers

University of Essex on 2026-04-24 <1%

21 Student papers

University of KwaZulu-Natal on 2019-10-03 <1%

22 Internet

www.mdpi.com <1%

23 Internet

digital.auraria.edu <1%

24 Internet

open.uct.ac.za <1%

25 Internet

semmelweis.hu <1%

26 Internet

www.sweetstudy.com <1%

27 Student papers

CITY College, University of York Europe Campus on 2026-02-21 <1%

28 Publication

Panditharatne, Dimuth V.. "Integrating Multimodal Sensors and Simulation Mode... <1%

29 Student papers

University of Moratuwa on 2013-10-18 <1%

30 Internet

www.preprints.org <1%

31 Student papers

Dayananda Sagar University, Bangalore on 2026-04-23 <1%

32 Student papers

Obudai Egyetem on 2025-12-16 <1%

33 Student papers

Royal Melbourne Institute of Technology on 2011-01-19 <1%

34 Student papers

Sri Lanka Institute of Information Technology on 2021-10-07 <1%

35 Internet

researcharchive.vuw.ac.nz <1%

36 Internet

web.wpi.edu <1%

37 Internet

www.springerprofessional.de <1%

38 Publication

Altarazi, Farah. "Integrating Process Mining, Robotic Process Automation, and Da... <1%

39 Student papers

CSU, San Jose State University on 2025-12-18 <1%

40 Publication

Mahadeo, Melissa Ramesh. "The Impact of Trust in Transformation Leadership of ... <1%

41 Student papers

Sri Lanka Institute of Information Technology on 2021-10-08 <1%

42 Student papers

University of Essex on 2026-04-24 <1%

43 Student papers

University of Ulster on 2012-05-04 <1%

44 Internet

ceur-ws.org <1%

45 Internet

epdf.tips <1%

46 Internet

ucalgary.scholaris.ca <1%

Leveraging Agentic AI Middleware for Intelligent Interaction and Modernization of Legacy Enterprise Applications

Project ID: 25-26J-455

3

BSc (Hons) Degree in Information Technology

Specializing in Software Engineering

Department of Software Engineering, Faculty of Computing

Sri Lanka Institute of Information Technology

Sri Lanka

April 2026

Leveraging Agentic AI Middleware for Intelligent Interaction and Modernization of Legacy Enterprise Applications

Project ID: 25-26J-455

1

Dissertation Submitted In Partial Fulfillment of the Requirements for the Bachelor of Science (Hons) Degree in Information Technology Specializing In Software Engineering

Department of Software Engineering, Faculty of Computing

Sri Lanka Institute of Information Technology

Sri Lanka

April 2026

DECLARATION

We declare that this is our own work, and this dissertation does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, we hereby grant to Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute our dissertation in whole or part in print, electronic or other medium. We retain the right to use this content in whole or part in future works (such as articles or books).

Name	Student ID	Signature
M Z M AYYASH	It22355850	
S M A C M RASHAD	It22331472	
M A M MUFTHI	It22355478	
D N RAJAPAKSHA	It22311436	

Mr. Vishan Jayasinghearachchi	Supervisor	
Dr. Dharshana Kasthurirathna	Co-Supervisor	

Date: …….

ABSTRACT

Enterprise organizations across healthcare, finance, and government continue to operate mission-critical software on decades-old desktop platforms, spending over $300 billion annually on maintenance while remaining unable to modernize due to source-code inaccessibility and prohibitive replacement costs. These legacy systems, built on technologies such as Visual Basic 6, COBOL, and PowerBuilder, are deeply embedded in daily operations and cannot be replaced without significant risk of operational disruption. Conventional modernization approaches re-hosting, re-platforming, and full re-engineering carry high costs, long timelines, and failure rates approaching 80%. Robotic Process Automation (RPA) provides a non-intrusive alternative but is crippled by static selector fragility, the inability to decompose high level goals, and no mechanism for intelligent self-recovery.

This report presents LAMA (Legacy Application Modernization Architecture), a comprehensive agentic AI middleware framework that modernizes legacy Windows desktop applications through non-intrusive orchestration, requiring zero modification to the underlying system. LAMA integrates four tightly coupled components operating as a coordinated pipeline. The Frontend Builder employs large language model (LLM) visual reasoning to automatically construct modern, accessible React web interfaces from legacy screenshots, eliminating the physical access barrier. SOACRS (Self-Optimizing Agent Coordination and Response System) is a lightweight Node.js microservice that generates failure-aware execution plans using a hybrid scoring engine combining rule-based capability matching with contextual bandit learning. The Orchestrator Agent executes these plans as directed acyclic graphs (DAGs), coordinating five specialized RPA bots through a blackboard-style shared memory architecture with pre- and post-condition verification at every step. The Low-Code RPA Connector interacts physically with the legacy GUI using CNN-based element detection, adaptive semantic macro replay across three input injection strategies, and a digital-twin simulator for offline validation, while exposing approved workflows as REST and GraphQL APIs.

The framework was evaluated against a Visual Basic 6 hospital patient-management system running on a Windows XP virtual machine across 150 executions spanning three clinical workflows. Results demonstrate an overall task success rate of 97.0%, an OCR field-level accuracy of 94.1%, and a workflow recovery rate of 98.8%, with a mean end-to-end execution time of 7.8 seconds. All but three failures recovered autonomously through retry and fallback mechanisms, with no execution requiring full compensatory rollback. The contextual bandit model demonstrated autonomous adaptation to a systematic vision-detection error without human intervention. LAMA offers healthcare, finance, and government organizations a practical, low-risk modernization pathway that preserves operational continuity while immediately delivering modern integration, automation, and accessibility capabilities.

Keywords: Legacy system modernization, Robotic process automation, Agentic AI, Large language models, Multi-agent systems.

ACKNOWLEDGEMENT

The successful completion of this research project would not have been possible without the guidance, support, and encouragement of numerous individuals to whom we are deeply grateful.

We extend our sincere gratitude to our supervisor, Mr. Vishan Jayasinghearachchi, for his invaluable guidance, constructive feedback, and continuous support throughout the research process. His expertise in software engineering and intelligent systems provided the foundation upon which this work was built. His patience and commitment to excellence have been a constant source of motivation.

We are equally grateful to our co-supervisor, Dr. Dharshana Kasthurirathna, whose deep knowledge of complex systems, network science, and research methodology shaped the theoretical foundations of this study. His insightful critiques and encouragement to pursue rigor in experimental design significantly improved the quality of this report.

We would like to thank the Department of Software Engineering at the Sri Lanka Institute of Information Technology (SLIIT) for providing the resources, facilities, and academic environment necessary to conduct this research. The support of the faculty and administrative staff has been instrumental.

We acknowledge the open-source community for the tools and frameworks that made this research possible, including the contributors to PyAutoGUI, AutoHotkey, OpenCV, Tesseract, React, Node.js, and the broader ecosystem of AI and machine learning libraries.

Finally, we express our heartfelt thanks to our families and friends for their unwavering support, patience, and encouragement throughout the demanding process of producing this research. Their belief in our work has been our greatest motivation.

TABLE OF CONTENTS

DECLARATION **I**

ABSTRACT **II**

ACKNOWLEDGEMENT **IV**

LIST OF FIGURES **IX**

LIST OF TABLES **IX**

LIST OF ABBREVIATIONS **X**

1. INTRODUCTION **1**

1.1 BACKGROUND 1

1.1.1 The legacy system problem in enterprise environments 1

1.1.2 Limitations of conventional modernization strategies 2

1.1.3 The emergence of agentic AI as a modernization pathway 3

1.2 LITERATURE REVIEW 3

1.2.1 Legacy system modernization approaches 3

1.2.2 Robotic process automation and gui automation 4

1.2.3 Ai-driven gui understanding and computer vision 5

1.2.4 Large language models in ui generation and task planning 5

1.2.5 Multi-agent system architectures and coordination protocols 6

1.2.6 BDI planning and cognitive agent architectures 6

1.2.7 Adaptive decision making and contextual bandit models 7

1.2.8 Reliability patterns in distributed systems 7

1.3 RESEARCH GAP 8

1.4 RESEARCH PROBLEM 8

1.5 RESEARCH OBJECTIVES 9

1.5.1 Main objective 9

1.5.2 Specific objectives 9

1.6 SCOPE AND LIMITATIONS 10

2. METHODOLOGY **11**

2.1 OVERALL SYSTEM ARCHITECTURE 11

2.1.1 Component interaction and data flow 14

2.1.2 Design principles and technology stack 15

2.2 FRONTEND BUILDER - DYNAMIC UI GENERATION COMPONENT 15

2.2.1 Component overview and role 15
2.2.2 Legacy screenshot capture and LLM-based extraction 16
2.2.3 React interface code generation and websocket integration 16
2.3 SOACRS – Self Optimizing Agent Coordination and Response System 17
2.3.1 Component overview and role 17
2.3.2 Hybrid scoring engine 18
2.3.3 BDI-Inspired guarded task planning 18
2.3.4 Sagas-pattern compensation and state machine generation 18
2.3.5 Semantic experience memory 19
2.4 Orchestrator Agent – Multi Agent Workflow Execution Component 19
2.4.1 Component overview and dag-based execution 19
2.4.2 Specialized agent pool 20
2.4.3 Blackboard-style shared memory 20
2.4.4 Three-tier failure handling 21
2.5 Low-Code RPA Connector - Legacy GUI Interaction Component 21
2.5.1 Component overview 21
2.5.2 AI-driven screen state classification and form recognition 21
2.5.3 Adaptive RPA macro recording and semantic replay 22
2.5.4 Digital twin simulator and rest/graphql api layer 23
2.6 Supporting Infrastructure 23
2.6.1 Windows XP virtualbox environment 24

3. COMMERCIALIZATION ASPECTS OF THE PRODUCT 24

3.1 Market Opportunity and Business Value 24
3.2 Target Industries and Use Cases 25
3.3 Commercialization Roadmap 25
3.4 Competitive Advantage 25

12 **4. TESTING AND EVALUATION 26**

4.1 Testing Strategy 26
4.2 Unit Testing 26
4.3 Integration Testing 27
4.3.1 Frontend to SOACRS integration 27
4.3.2 SOACRS to orchestrator integration 27
4.3.3 Orchestrator to rpa connector integration 28
21 4.4 System Testing – End to End Clinical Workflows 28
4.4.1 Patient list retrieval workflow 28
4.4.2 Add new patient workflow 28
4.4.3 Remove patient workflow 29

17

4.5 EVALUATION METRICS 29

5. IMPLEMENTATION 30

5.1 DEVELOPMENT METHODOLOGY AND TOOLS 30
5.2 IMPLEMENTATION - FRONTEND BUILDER 30
5.3 IMPLEMENTATION – SOACRS 31
5.4 IMPLEMENTATION - ORCHESTRATOR AGENT 31
5.5 IMPLEMENTATION - LOW-CODE RPA CONNECTOR 31
5.6 SYSTEM INTEGRATION AND END-TO-END WORKFLOW ASSEMBLY 32
5.7 TARGET LEGACY SYSTEM CONFIGURATION 32
5.7.1 Visual basic 6 hospital management system 32

23

6. RESULTS AND DISCUSSION 33

6.1 QUANTITATIVE RESULTS 33
6.1.1 Task success rate analysis 33
6.1.2 Execution time analysis 34
6.1.3 OCR field-level accuracy 34
6.1.4 Workflow recovery and failure analysis 34
6.2 CONTEXTUAL BANDIT CONVERGENCE AND SELF-OPTIMIZATION 35
6.3 HYBRID INPUT INJECTION STRATEGY COMPARISON 36
6.4 DIGITAL TWIN SIMULATOR EFFECTIVENESS 37
6.5 FEATURE COMPARISON WITH RELATED APPROACHES 37
6.6 RESEARCH FINDINGS 38
6.6.1 Frontend builder 38
6.6.2 SOACRS - self optimizing agent coordination and response system 38
6.6.3 Orchestrator agent 39
6.6.4 Low-code rpa connector 39
6.7 DISCUSSION 39
6.7.1 Overall system performance 39
6.7.2 Architectural strengths 40
6.7.3 Limitations 40
6.7.4 Threats to validity 41
6.8 SUMMARY OF EACH STUDENT'S CONTRIBUTION 41

7. CONCLUSION 44

5

7.1 SUMMARY OF CONTRIBUTIONS 44
7.2 ANSWERS TO RESEARCH OBJECTIVES 44
7.3 FUTURE WORK 45

REFERENCES 46

APPENDICES **49**

APPENDIX A: TARGET LEGACY SYSTEM SCREENSHOTS 49

APPENDIX B: FRONTEND BUILDER (BUILDER APP) SCREENSHOTS 52

APPENDIX C: SOACRS SCORING FUNCTION CONFIGURATION 55

APPENDIX D: RPA BOT REGISTRATION SCHEMA 56

APPENDIX E: SAMPLE EXECUTION PLAN JSON 56

APPENDIX F: TEST CASE SPECIFICATIONS 57

APPENDIX G: EVALUATION DATASET DESCRIPTION 57

3

LIST OF FIGURES

Figure 2.1: High-Level System Architecture of the LAMA Framework 11
Figure 2.2: Execution Flowchart 12
Figure 2.3: SOACRS Five-Stage Processing Pipeline 17
Figure A.1: HIS-VB6 Main Dashboard and Navigation Screen 49
Figure A.2: Add Patient Form Screen 50
Figure A.3: Patient List with Delete Patient Dialog 50
Figure A.4: Treatment History Screen 51
Figure A.5: Add Treatment Details Form 51

LIST OF TABLES

Table 2.1: LAMA Technology Stack by Component 15
Table 2.2: Specialized RPA Agent Pool: Responsibilities and Mechanisms 20
Table 4.1: Unit Test Coverage by Component 27
Table 6.1: Evaluation Results Across Three Clinical Workflows (50 Trials Each) 33
Table 6.2: Failure Analysis by Workflow Category 34
Table 6.3: Input Injection Strategy Comparison 36
Table 6.4: Feature Comparison with Related Approaches 37
Table 6.5: Summary of Each Student's Contribution 41

LIST OF ABBREVIATIONS

Abbreviation	Description
AI	Artificial Intelligence
API	Application Programming Interface
BDI	Belief-Desire-Intention
CNN	Convolutional Neural Network
CSS	Cascading Style Sheets
DAG	Directed Acyclic Graph
ELK	Elasticsearch, Logstash, Kibana
GUI	Graphical User Interface
HIS	Hospital Information System
JSON	JavaScript Object Notation
LAMA	Legacy Application Modernization Architecture
LLM	Large Language Model
MAS	Multi-Agent System
NLP	Natural Language Processing
OCR	Optical Character Recognition
OS	Operating System
REST	Representational State Transfer
RPA	Robotic Process Automation
SLA	Service Level Agreement
SOACRS	Self-Optimizing Agent Coordination and Response System
UI	User Interface
UUID	Universally Unique Identifier
VB6	Visual Basic 6
VM	Virtual Machine
WCAG	Web Content Accessibility Guidelines

WSS	WebSocket Secure
XML	Extensible Markup Language
XP	Experience (Windows XP)

LIST OF APPENDICES

Appendix A - Target Legacy System Screenshots 49
Appendix B - Frontend Builder (Builder App) Screenshots 52
Appendix C - SOACRS Scoring Function Configuration 55
Appendix D - RPA Bot Registration Schema 56
Appendix E - Sample Execution Plan JSON 56
Appendix F - Test Case Specifications 57
Appendix G - Evaluation Dataset Description 57

24

1. INTRODUCTION

1.1 Background

1.1.1 The legacy system problem in enterprise environments

Legacy information systems have formed the operational backbone of critical industries for several decades. In healthcare, finance, logistics, public administration, and manufacturing, these systems manage essential processes ranging from patient record management and banking transactions to government service delivery and supply chain tracking. Despite being built on technologies that are decades old such as COBOL, Visual Basic 6, PowerBuilder, and early client-server frameworks legacy systems are remarkably persistent. Their longevity stems from three compounding factors: proven operational stability cultivated over decades of real-world use, deep embedding in organizational workflows that have evolved around them, and the prohibitive cost and risk associated with replacing them.

The scale of this dependency is enormous. A 2023 Gartner analysis estimated that organizations collectively spend over $300 billion per year maintaining these systems, with the majority of that expenditure directed toward preventing failures rather than adding new capabilities [1]. Industry surveys consistently report that 70 to 75 percent of global IT budgets are consumed by maintaining existing legacy infrastructure, leaving minimal resources for innovation, security hardening, or digital transformation initiatives [28].

The consequences extend beyond financial burden. Legacy systems typically expose only proprietary, GUI-only interfaces and contain no programmatic integration hooks such as REST APIs, database connectors, or service-oriented endpoints. This architectural rigidity creates severe barriers for organizations seeking to adopt modern cloud platforms, mobile applications, real-time analytics tools, or AI-driven automation. Staff in hospitals or government offices must be physically present at a specific machine running a specific operating system to perform even routine tasks.

13

The 2017 Equifax data breach, which exposed the personal information of approximately 147 million people, was rooted in vulnerabilities in a platform originally developed in the 1970s, demonstrating the real-world security consequences of unresolved legacy system vulnerabilities.

1.1.2 Limitations of conventional modernization strategies

Organizations confronting legacy system constraints typically consider one of three conventional modernization strategies re-hosting, re-platforming, or full re-engineering. Re-hosting (lift-and-shift) migrates the application to new infrastructure with minimal code changes. While this improves operational efficiency, it preserves the underlying architectural rigidity. Re-platforming adapts the application to a new runtime environment but may introduce data consistency risks and typically requires partial code modification.

Full re-engineering complete redesign using modern languages and architectures offers the greatest long-term flexibility but is prohibitively expensive and risky. Market analyses report that modernization projects average 16 months in duration, cost approximately $1.5 million, and have failure or abandonment rates approaching 80% [25]. For organizations where downtime is unacceptable, where the original codebase is inaccessible or undocumented, or where the specialized developers who built the system are no longer available, these approaches are neither practical nor sustainable.

Robotic Process Automation (RPA) emerged as a promising non-intrusive alternative. By simulating human GUI interactions, RPA tools such as UiPath, Automation Anywhere, and Blue Prism enable automated workflows without source code modification. However, the fundamental limitation of conventional RPA is well documented: bots rely on static selectors such as absolute screen coordinates, element identifiers, or pixel-level image matching. Even minor changes to the interface layout a phenomenon known as UI drift can break automations ([3], [4]). Furthermore, traditional RPA frameworks lack the intelligence to interpret high-level operational goals, decompose them into subtasks, select among competing execution strategies, or self-recover from unexpected system states.

1.1.3 The emergence of agentic AI as a modernization pathway

Recent advances in agentic AI autonomous agents capable of planning, acting, monitoring, and self-correcting in dynamic, partially observable environments open a fundamentally new design space for legacy modernization. Large Language Models (LLMs) such as GPT-4 and Meta LLaMA have demonstrated the ability to understand and reason about visual interface content, extract structured information from screenshots, and generate functional code from high-level specifications. Computer vision frameworks have matured to the point where GUI elements can be detected and classified from screenshots with high accuracy [12]. Multi-agent coordination architectures provide sophisticated mechanisms for decomposing complex goals into atomic subtasks, selecting among specialized agents based on real-time performance data, and orchestrating their execution with formal correctness guarantees [20], [21].

The convergence of these capabilities makes it now feasible to build a middleware layer that sits above an unchanged legacy application, generates a modern interface from visual analysis alone, constructs intelligent execution plans that adapt to runtime conditions, orchestrates multiple specialized automation agents, and exposes the resulting capabilities as programmable modern services all without requiring access to the legacy system source code, database, or operating environment. This is the design space that the LAMA framework occupies.

1.2 Literature Review

1.2.1 Legacy system modernization approaches

Bennett [2] established the foundational challenge: the more reliably a system performs its core function, the more deeply it embeds into surrounding processes and the harder it becomes to replace a phenomenon he termed the paradox of success. This observation, made three decades ago, remains as accurate today as when it was first articulated. Kazman et al. [5] subsequently developed architecture reconstruction techniques that map hidden dependencies without requiring full re-engineering,

enabling organizations to identify modular boundaries, characterize technical debt, and plan targeted interventions rather than wholesale replacements.

Perez-Castillo et al. [6] demonstrated screen scraping as a practical non-intrusive extension mechanism for mainframe systems in regulated environments, presenting a case study of extending a hospital information system without code changes. Their work validated the feasibility of GUI-level integration in compliance-heavy contexts and established the technical precedent upon which modern RPA and agentic middleware approaches are built.

1.2.2 Robotic process automation and gui automation

Lacity and Willcocks [3] conducted one of the most comprehensive analyses of commercial RPA deployment across multiple enterprises in different industries. Their findings confirmed that RPA provides measurable productivity improvements for rule-based, repetitive GUI tasks. However, they specifically identified selector fragility the tendency of hard-coded element locators to break under interface updates as the dominant long-term operational challenge, noting that maintenance burdens often approach or exceed the cost of the manual operations they replaced.

Ribeiro et al. [7] conducted a systematic literature review of 52 studies on AI-enhanced RPA. Their findings revealed that integrating computer vision and machine learning into automation pipelines significantly improved resilience to UI drift compared with purely selector-based approaches. However, even AI-enhanced RPA in the reviewed literature operated primarily as standalone automation scripts without an intelligent coordination layer capable of managing multiple bots across multi-step workflows. El-Gharib et al. [8] demonstrated that process mining analysis of event logs from legacy workflows can identify optimal automation targets before RPA deployment, improving rollout efficiency. Oberhauser and Pogolski [4] explored partially self-healing RPA capable of recovering from minor layout changes without human

intervention, though their framework lacked higher-level task planning, multi-agent coordination, or API integration capabilities.

1.2.3 Ai-driven gui understanding and computer vision

Beltramelli [9] established the feasibility of automated GUI analysis without framework access, achieving 77% accuracy generating multi-platform UI code from GUI screenshots using a combined CNN-LSTM architecture. Li et al. [10] provided the RICO dataset 66,261 annotated Android application screenshots across 27 categories of UI metadata enabling the training of specialized models for UI element recognition at scale. Luo et al. [11] advanced the state of the art with OmniParser, which parses arbitrary screenshots into structured element trees suitable for reliable action grounding. Wang et al. [12] demonstrated autonomous GUI agents operating entirely through native interfaces via visual understanding alone with UI-TARS. UI-TARS is the closest prior work to the LAMA RPA connector; the key differentiator is that UI-TARS operates as a standalone agent, while the LAMA connector is one component in a coordinated multi-agent pipeline with API-exposure capabilities and offline digital-twin validation.

1.2.4 Large language models in ui generation and task planning

The application of large language models to user interface generation represents a rapidly developing research area. Multimodal LLMs such as GPT-4V and Meta LLaMA Vision can reliably extract structured descriptions of interface components from screenshots, including element types, labels, data types, and spatial relationships. This capability is central to the LAMA Frontend Builder, which uses LLM reasoning to bridge the gap between unstructured legacy interface screenshots and structured JSON schemas suitable for code generation. Ramalingam [29] demonstrated that LLM-driven agents can be integrated into enterprise application integration frameworks to automate workflows, reduce latency, and adapt dynamically to system constraints.

1.2.5 Multi-agent system architectures and coordination protocols

Smith [13] established one of the most influential models for decentralized task allocation in multi-agent systems through the Contract Net Protocol (CNP). In CNP, a manager agent announces available tasks, contractor agents submit bids based on their capabilities and workload, and the manager awards the task to the most suitable bidder. The LAMA orchestrator agent capability filtering mechanism draws directly from CNP principles. Nii [14] introduced the Blackboard Architecture an alternative coordination paradigm based on shared global state where multiple independent knowledge sources contribute partial solutions to a centralized data structure. The LAMA orchestrator incorporates blackboard-style shared memory implemented as Redis hash structures to enable flexible inter-agent information sharing during execution.

Lim et al. [20] demonstrated hierarchical orchestration in AgentOrchestra, where a conductor agent decomposes high-level objectives into subtasks and coordinates domain-specialized agents. Gupta and Sharma [21] introduced DynTaskMAS dynamic task graphs and asynchronous scheduling achieving near-linear scalability when parallelizing independent task branches. Both systems validate the DAG-based execution model used in the LAMA orchestrator.

1.2.6 BDI planning and cognitive agent architectures

The Belief-Desire-Intention (BDI) model, formalized for software agents by Rao and Georgeff [16], [18] drawing on Bratman's (1987) philosophical work on practical reasoning, provides the theoretical foundation for goal-directed planning with dynamic plan revision that underlies SOACRS. In the BDI model, an agent's mental state is characterized by its beliefs about the environment, its desires representing goals it wants to achieve, and its intentions representing the plans it has committed to executing. Meneguzzi and de Silva [17] surveyed planning methods that integrate symbolic BDI reasoning with real-time execution, demonstrating that agents can adapt their plans dynamically as beliefs and desires change.

1.2.7 Adaptive decision making and contextual bandit models

Li et al. [19] introduced contextual bandit algorithms as a tool for online adaptive decision-making, demonstrating their effectiveness for balancing exploration of new strategies against exploitation of known-successful policies based on real-time context features. The LinUCB formulation they developed provides a computationally efficient online learning algorithm that maintains a linear model of expected reward as a function of context features, with an uncertainty bonus that encourages exploration of under-evaluated options. This framework is directly applied in the SOACRS scoring engine to enable adaptive tool selection that improves continuously from execution experience without requiring offline retraining. Dragone et al. [27] extended contextual bandits to incorporate personalized reward signals, demonstrating improved performance in dynamic recommendation environments.

1.2.8 Reliability patterns in distributed systems

Garcia-Molina and Salem [22] introduced the Sagas model, which addresses long-running transaction management by decomposing transactions into a sequence of atomic sub-transactions, each with a corresponding compensatory action that restores system consistency if the sub-transaction fails. The Sagas pattern is particularly applicable to RPA workflows, where each GUI interaction step may succeed or fail independently and recovery requires undoing or retrying specific actions rather than rolling back the entire workflow. The SOACRS plan constructor specifies Sagas-compatible compensation paths for each execution step.

Sigelman et al. [23] developed the Dapper distributed tracing system at Google, which assigns trace IDs to requests, propagates them through distributed service calls, and records timing and causality information at each processing step. Incorporating Dapper-style tracing through OpenTelemetry in LAMA ensures that every agent delegation decision, validation outcome, and compensation action is recorded with sufficient context for compliance auditing and performance debugging a critical requirement for deployment in regulated healthcare and financial environments.

1.3 Research Gap

The literature survey reveals a consistent pattern of fragmented progress across the domains relevant to legacy system modernization. RPA approaches provide non-intrusive access to legacy GUIs but lack intelligence, self-optimization, and coordination capabilities. AI-driven GUI understanding improves interaction resilience but operates in isolation without integration planning or API exposure. Multi-agent coordination architectures provide sophisticated task allocation and learning mechanisms but have not been applied to GUI-mediated legacy integration environments.

Specifically, no existing system combines: (1) automated modern UI generation from legacy screenshots without source-code access; (2) BDI-inspired adaptive planning with contextual bandit delegation and semantic experience memory; (3) DAG-based orchestration with blackboard shared memory and Sagas-pattern compensation; and (4) CNN-based GUI element detection with adaptive macro recording, digital-twin offline simulation, and REST/GraphQL API exposure all within a single, production-grade middleware framework that operates without any modification to the legacy system. The LAMA framework was designed specifically to fill this compound gap.

1.4 Research Problem

The central research problem addressed by this work can be stated as follows: How can an intelligent middleware system be designed and implemented to modernize legacy GUI-only enterprise applications, enabling modern web-based user interaction, adaptive workflow automation, and programmatic API access, without requiring any modification to the legacy application's source code, database, or operational environment?

This problem encompasses five interrelated challenges. First, the interface accessibility challenge: legacy GUI interfaces are accessible only through specific hardware and OS configurations, excluding remote users and modern devices. Second,

the integration fragility challenge: conventional RPA automation breaks under UI drift and cannot adapt to variable runtime conditions. Third, the intelligent planning challenge: complex multi-step workflows require decomposition, agent selection, and failure recovery capabilities exceeding what static automation scripts can provide. Fourth, the execution reliability challenge: interacting with legacy applications inside virtualized environments introduces additional unpredictability in keyboard and mouse event handling. Fifth, the programmability challenge: legacy operations must be exposed as modern, secure, versioned API endpoints to enable integration with contemporary digital systems.

1.5 Research Objectives

1.5.1 Main objective

To design and implement LAMA, an agentic AI middleware framework that modernizes legacy GUI-only enterprise applications by providing modern web interface generation, intelligent multi-agent workflow orchestration, and REST/GraphQL API exposure, without requiring any modification to the legacy application's source code, database, or operational environment.

1.5.2 Specific objectives

1. Develop a Frontend Builder component that leverages LLM visual reasoning to automatically generate accessible React web interfaces from legacy application screenshots, with real-time bidirectional communication via WebSocket.
2. Design and implement SOACRS, a plan-construction microservice that combines deterministic rule-based capability matching with contextual bandit learning to generate failure-aware execution plans with BDI-inspired pre/post-condition verification and Sagas-pattern compensation paths.

3. Build an Orchestrator Agent that models execution plans as DAGs, coordinates five categories of specialized RPA agents through blackboard-style shared memory, enforces pre- and post-condition verification at every step, and implements a three-tier failure handling hierarchy.
4. Implement a Low-Code RPA Connector that uses CNN-based GUI element detection, adaptive semantic macro recording and replay with a three-strategy input injection chain, a digital-twin simulator for offline validation, and REST/GraphQL API exposure for programmatic access to legacy workflows.
5. Evaluate the complete LAMA system against a VB6 hospital management application in a Windows XP virtual machine environment, measuring task success rate, execution time, OCR accuracy, and workflow recovery rate across 150 trials.

29

1.6 Scope and Limitations

The scope of this research encompasses the design, implementation, and empirical evaluation of the LAMA framework applied to Windows-based legacy desktop applications. The evaluation target is a Visual Basic 6 hospital patient-management system, providing a realistic worst-case modernization scenario. The three workflows evaluated patient list retrieval, patient registration, and patient removal represent the core data management operations common to most legacy enterprise applications.

The following limitations apply to the current implementation. The RPA connector depends on Windows-only libraries (PyAutoGUI, PyDirectInput, AutoHotkey), restricting the middleware host to Windows environments. OCR accuracy of 94.1%, while adequate for demonstration, may be insufficient for production healthcare deployments where patient record fidelity is regulated. The contextual bandit model requires approximately ten trials on a given field type before selections converge, representing reduced performance during initial deployment. The current implementation has not undergone formal penetration testing or regulatory compliance review for HIPAA or PCI-DSS.

2. METHODOLOGY

2.1 Overall System Architecture

LAMA modernizes legacy desktop applications without modifying their source code. The framework begins when an administrator captures a screenshot of the legacy application's active interface. An LLM processes this image to extract a structured JSON (JavaScript Object Notation) description of the screen's components. That JSON feeds a code-generation pipeline producing a modern React web interface, which users interact with from any device or operating system. User inputs travel via WebSocket to the SOACRS planning service, which generates an ordered execution plan. The plan is handed to the orchestrator, which dispatches RPA bots to execute it against the legacy system running in a background VM. Figure 2.1 illustrates the high-level system architecture. Figure 2.2 shows the execution flowchart

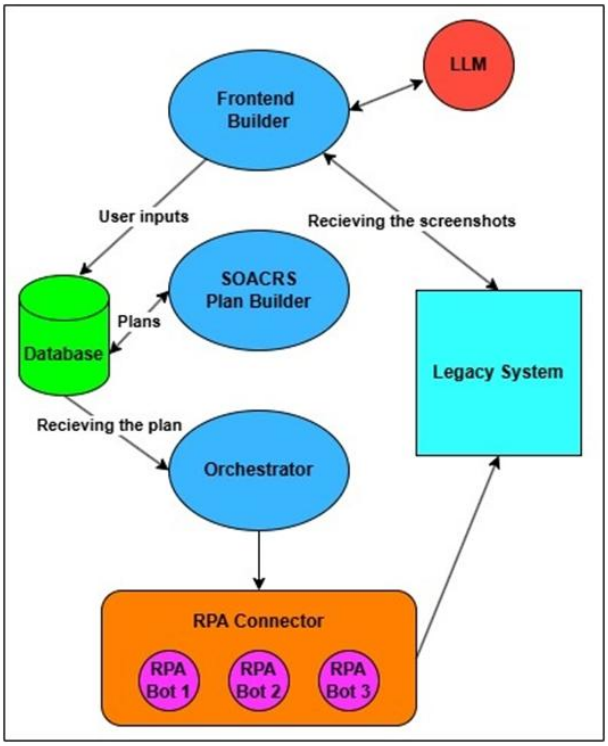

Figure 2.1: High-Level System Architecture of the LAMA Framework

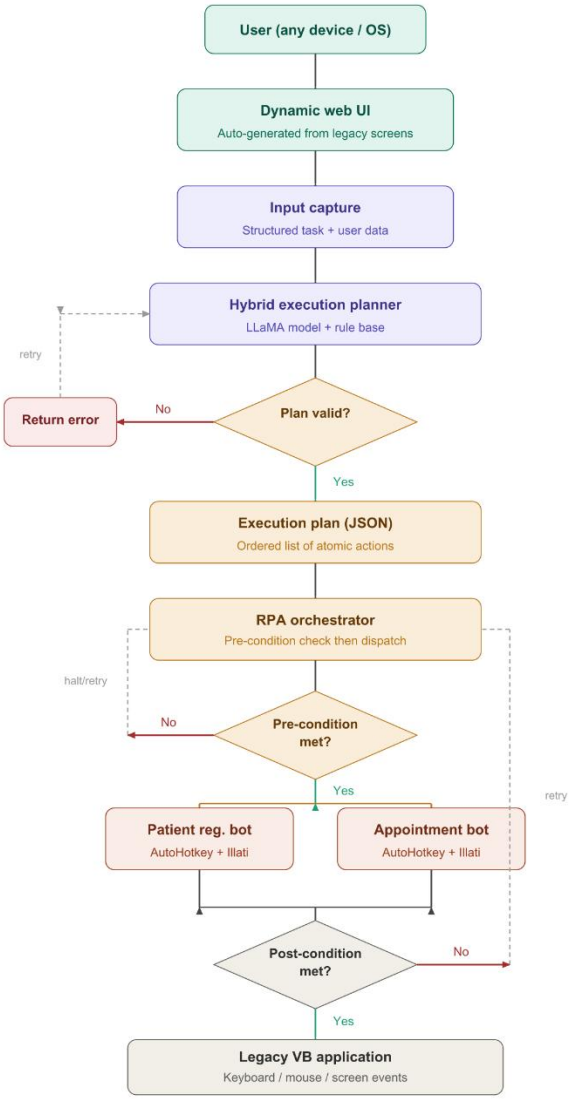

Figure 2.2: Execution Flowchart

As illustrated in Figure 2.1, the LAMA framework is organized around four principal components that communicate through well-defined interfaces. The Frontend Builder occupies the uppermost position in the pipeline, receiving screenshots of the legacy system (labelled "Receiving the screenshots") from the Legacy System module on the right and interacting with the LLM (Meta LLaMA 3.1 70B or GPT-4V, shown at top right) to perform structured JSON extraction. User inputs received by the Frontend Builder are passed downward to the SOACRS Plan Builder, which also reads from and writes to the shared Database on the left. SOACRS constructs a complete execution plan and passes it (labelled "Receiving the plan") to the Orchestrator, which

coordinates the RPA Connector below it. The RPA Connector houses three specialized RPA bots (Bot 1, Bot 2, Bot 3) that interact directly with the Legacy System through keyboard, mouse, and screen events. This top-to-bottom flow separates interface generation (Frontend Builder), intelligent planning (SOACRS), workflow coordination (Orchestrator), and physical GUI interaction (RPA Connector) into discrete, independently maintainable layers.

Figure 2.2 presents the end-to-end execution flowchart of the LAMA framework, tracing the complete path from a user request on any device or operating system through to final interaction with the legacy Visual Basic application. The flow begins at the top with the User (any device/OS) accessing the Dynamic Web UI, which is auto-generated from legacy screens by the Frontend Builder. The Input Capture stage serializes the user interaction as a structured task description with accompanying field data, which is forwarded to the Hybrid Execution Planner, a module combining the LLaMA language model with a deterministic rule base to produce a validated execution plan.

The plan validation decision diamond in the center of the flowchart introduces the first self-correction loop: if the plan is not valid, a retry is triggered back to the Hybrid Execution Planner; if the plan repeatedly fails validation, the system returns an error. Once a valid plan is produced, it is serialized as an Execution Plan (JSON) an ordered list of atomic actions and handed to the RPA Orchestrator, which performs a Pre-Condition Check before dispatching any bot. This pre-condition gate is the critical mechanism that prevents the common RPA failure mode of attempting to interact with screen elements that do not exist in the current application state. If the pre-condition is not met (the legacy system is on an unexpected screen), the orchestrator issues a halt/retry rather than proceeding blindly.

When the pre-condition is satisfied, the Orchestrator dispatches the appropriate specialized bot in the evaluated hospital management scenario, either the Patient Registration Bot or the Appointment Bot, each using AutoHotkey combined with low-

level input injection. After bot execution, the Post-Condition Check diamond verifies that the expected outcome has been achieved (for example that a newly registered patient appears in the patient list). If the post-condition is not met, a retry is triggered; only when post-condition verification passes does control flow downward to the Legacy VB Application (Keyboard/Mouse/Screen Events layer), confirming that the complete interaction cycle has been successfully executed. This double-verification architecture pre-condition before dispatch and post-condition after execution is what distinguishes LAMA from conventional RPA, which executes fixed scripts without correctness guarantees at any point in the workflow.

2.1.1 Component interaction and data flow

The end-to-end data flow through LAMA begins when an administrator uses the Frontend Builder to scan the legacy application. A screenshot is captured via PyAutoGUI and submitted to an LLM for structured JSON extraction. The JSON is processed by the code-generation pipeline to produce a React web application, which is served to users. When a user submits a form or triggers an action in the modern interface, the event is serialized as a structured JSON object containing action type, field values, session token, and a trace UUID. This event is transmitted via WebSocket to SOACRS.

SOACRS processes the event through its intent recognition module, generates a Task Specification Object (TSO), applies its scoring engine to select optimal agents, runs the guarded task planner to construct pre/post-condition-annotated execution steps, and emits the completed execution plan as a JSON object to the Orchestrator via Redis Streams. The Orchestrator parses the DAG structure, performs a pre-condition check by capturing and analyzing a screenshot of the current legacy system state, and dispatches the appropriate RPA bot. After each step, a post-condition check verifies the expected outcome. Results are aggregated and returned to the frontend via WebSocket, with intermediate status updates pushed in real time.

2.1.2 Design principles and technology stack

The LAMA framework was designed according to five core principles: non-intrusiveness (zero modification to the legacy system), modularity (each component independently deployable and testable), resilience (multiple fallback strategies at every level), observability (every decision and outcome logged and traceable), and programmability (legacy operations exposed as modern APIs).

Table 2.1: LAMA Technology Stack by Component

Component	Primary Technologies	Supporting Infrastructure
Frontend Builder	React, Vite, Tailwind CSS, Zod, Socket.IO	GPT-4V / Meta LLaMA 3.1 70B API
SOACRS	Node.js 20, Express.js, TypeScript, LinUCB (custom)	PostgreSQL + pgvector, Redis Streams
Orchestrator Agent	Python 3.11, FastAPI, asyncio, NetworkX	Redis (blackboard), OpenTelemetry
RPA Connector	Python 3.11, PyAutoGUI, PyDirectInput, AutoHotkey, OpenCV, Tesseract 5.3	FastAPI, Strawberry-GraphQL, MongoDB
Authentication	Keycloak (OAuth2/OIDC), JWT validation	HashiCorp Vault
Observability	Prometheus, Grafana, OpenTelemetry	Elasticsearch, Logstash, Kibana
CI/CD & Dev	Git, GitHub Actions, Docker Compose	Visual Studio Code, PyCharm, Postman

2.2 Frontend Builder - Dynamic UI Generation Component

2.2.1 Component overview and role

The Frontend Builder is the entry point of the LAMA pipeline. It addresses the most visible barrier to legacy system usability: the requirement that users be physically present at a specific machine running a specific operating system. By generating a full

modern web interface from a screenshot, LAMA makes the legacy application accessible from any browser on any device without installing anything locally. The component operates in two phases: an offline generation phase triggered by an administrator, and an ongoing online event handling phase that captures all user interactions and forwards them to SOACRS.

2.2.2 Legacy screenshot capture and LLM-based extraction

Each target legacy screen is captured as a PNG image using PyAutoGUI's screenshot function, which accesses the VM window directly via the Windows GDI API. Captured images are preprocessed before LLM submission: they are converted to a standardized 1024×768 resolution, and contrast is enhanced using CLAHE (Contrast Limited Adaptive Histogram Equalization) to improve element visibility in environments with inconsistent rendering. The preprocessed image is encoded as a base64 string for inclusion in the LLM API request payload.

The preprocessed screenshot is submitted to an LLM (tested with Meta LLaMA 3.1 70B and GPT-4V) with a structured prompt requesting JSON extraction across five element categories: form fields with inferred labels, data types, and validation constraints; action buttons with labels and expected behavior; navigation controls; display elements; and workflow relationships between screens. The LLM infers data types from field labels "Date of Birth" maps to a date-picker input type, "Patient ID" is flagged as numeric and required, "Sex" maps to a select input with inferred options. Extractions below a configurable confidence threshold of 0.75 are marked for human review before the interface is deployed.

2.2.3 React interface code generation and websocket integration

The validated JSON schema is consumed by a code-generation pipeline that maps each extracted element category to its React equivalent. Form fields map to controlled React input components with appropriate HTML5 input types. Select elements are generated with inferred option sets. Action buttons map to styled button components with onClick handlers that serialize the interaction as a structured event object. Validation

constraints are implemented using the Zod schema validation library, providing real-time client-side validation feedback before form submission. The generated interface is styled using Tailwind CSS following WCAG 2.1 AA accessibility guidelines.

Every user action on the generated interface triggers a structured JSON event object containing the action type, field values, a session token, and a trace UUID transmitted via Secure WebSocket to SOACRS. Real-time execution status updates are pushed from the Orchestrator back to the frontend via the same WebSocket connection, including workflow initiation confirmation, per-step progress indicators, extraction results, and completion or error notifications.

2.3 SOACRS – Self Optimizing Agent Coordination and Response System

2.3.1 Component overview and role

SOACRS is a lightweight Node.js microservice that acts as the cognitive core of the LAMA pipeline. Its role is narrow but critical: it transforms a structured user event into a complete, failure-aware execution plan specifying which agents should perform which actions, in what order, under what pre-conditions, and with what compensation strategies on failure. SOACRS does not execute any actions itself; it is purely a planning and routing service. This separation of planning from execution enables SOACRS to be developed, tested, and updated independently of the execution layer.

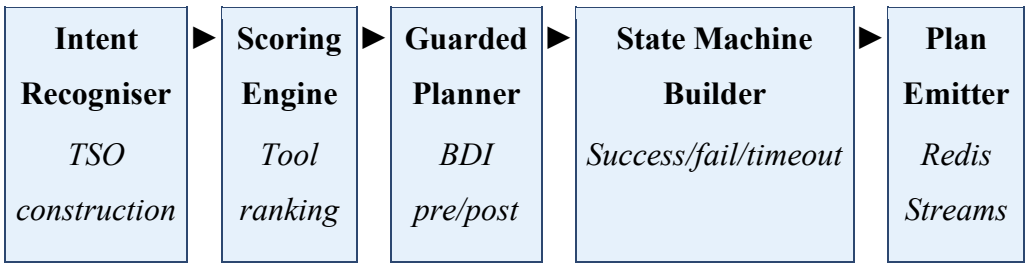

Figure 2.3: SOACRS Five-Stage Processing Pipeline

2.3.2 Hybrid scoring engine

The scoring engine evaluates each registered tool against four criteria for a given task request. The composite score for a candidate tool t is defined as,

$$S(t) = w_1 \cdot C(t) + w_2 \cdot R(t) + w_3 \cdot U(t) - w_4 \cdot K(t) \quad \text{... (Equation 2.1)}$$

where C(t) is a binary capability-match indicator (1 if the tool declares support for the required task type, 0 otherwise), R(t) is the tool's observed success rate over recent executions tracked as an exponential moving average, U(t) is the normalized mean user-satisfaction rating, and K(t) is the estimated execution cost (normalized and inverted so lower cost scores higher). The weights w_1–w_4 are configurable per deployment context with defaults of 0.4, 0.3, 0.2, and 0.1 respectively, reflecting the primacy of capability match over cost optimization. The highest-scoring eligible tool is selected as the primary agent; the second-highest becomes the designated fallback.

2.3.3 BDI-Inspired guarded task planning

The Guarded Task Planner converts each TSO into a step sequence with explicit pre-conditions and post-conditions using BDI-inspired reasoning [18], [17]. For the Add New Patient workflow, the plan specifies five steps with: the primary and fallback agents selected by the scoring engine; a pre-condition expressed as a screen-state assertion (e.g., "current screen must be Patient List"); a post-condition expressed as an outcome assertion (e.g., "Add Patient form must be open with all required fields visible"); a timeout in seconds; and a retry count. Pre-conditions are verified by the Orchestrator before dispatching the agent; post-conditions are verified after the agent reports completion.

2.3.4 Sagas-pattern compensation and state machine generation

SOACRS specifies Sagas-compatible compensation paths for each execution step at plan construction time [22]. Compensation paths are reverse sequences of recovery

actions that restore system consistency if a step fails after partially executing. The resulting execution plan is structured as a state machine. Each action carries explicit transitions for three outcomes: success (advance to the next step), failure (invoke the fallback agent or execute the compensation path and return an error), and timeout (retry up to a configured limit before treating the step as failed). This structure gives the Orchestrator a deterministic path to follow regardless of runtime conditions.

2.3.5 Semantic experience memory

SOACRS maintains a semantic experience memory using PostgreSQL with the pgvector extension. After each completed execution, a trace record is stored as a 768-dimensional vector embedding using the all-MiniLM-L6-v2 sentence transformer model. When a new task arrives, SOACRS performs a k-nearest-neighbor search (k=5) over the experience memory, retrieving the five most semantically similar historical executions. These recalled experiences serve two purposes: cold-start mitigation (providing informative priors for new agent types) and anomaly detection (flagging tasks with low similarity to historical executions for elevated monitoring). The experience memory grows continuously with each execution, progressively improving agent selection accuracy.

2.4 Orchestrator Agent – Multi Agent Workflow Execution Component

2.4.1 Component overview and dag-based execution

The Orchestrator Agent is the execution backbone of the LAMA framework, responsible for translating abstract execution plans from SOACRS into concrete, sequenced agent dispatches with real-time monitoring and failure handling. The execution plan received from SOACRS is parsed into a directed acyclic graph (DAG) where each node is an atomic execution step and directed edges represent dependency relationships between steps. The Orchestrator uses a topological sort algorithm to identify the "ready set" steps with all dependencies satisfied and dispatches them concurrently to available agents, enabling parallel execution of independent workflow branches.

2.4.2 Specialized agent pool

The Orchestrator manages a pool of five categories of specialized agents, each optimized for a specific class of interaction with the legacy GUI. These agents are Python processes that register with the Orchestrator at startup and communicate via the Orchestrator's RESTful agent management API. Table 2.2 summarizes each agent's responsibility and primary mechanism.

Table 2.2: Specialized RPA Agent Pool: Responsibilities and Mechanisms

Agent	Primary Responsibility	Key Mechanism
Navigation Agent	Screen transitions, menu navigation, dialog management	Relative-coordinate calibration (r_x, r_γ); screen-state classifier verification after each transition
Data Entry Agent	Form field population	Vision detection → coordinate fallback → AutoHotkey; clipboard paste for alphanumeric; SendInput for date pickers
Data Extraction Agent	OCR text extraction from display screens	Tesseract v5.3 LSTM; CLAHE preprocessing; line-height and column-boundary segmentation; MongoDB persistence
Validation Agent	Post-condition verification	Hue-based screen-state classification (98.7% accuracy); OCR content verification of submitted values
Monitoring Agent	Execution state observation; failure detection	OpenTelemetry spans; stall detection; R(t) reward aggregation for SOACRS bandit update

2.4.3 Blackboard-style shared memory

Inter-agent information sharing within a workflow execution is mediated through a blackboard-style shared memory implemented as a Redis hash structure keyed by the execution trace ID [14]. Each agent writes its output to the blackboard screen state, filled field records, OCR results, post-condition outcomes which downstream agents

read without direct inter-agent coupling. The blackboard is ephemeral, scoped to a single workflow execution, and archived to SOACRS's persistent experience memory upon completion.

2.4.4 Three-tier failure handling

The Orchestrator implements a three-tier failure handling hierarchy. At the first tier, transient failures are handled by step-level retry with exponential backoff: up to three retries with delays of 0.5, 1.0, and 2.0 seconds. At the second tier, persistent failures invoke the fallback agent strategy specified in the SOACRS plan. At the third tier, if all strategies for a step are exhausted, the Orchestrator invokes the Sagas-pattern compensation workflow, executing the reverse sequence of compensatory actions to restore system consistency before reporting the failure. Every failure event, retry attempt, and compensation invocation is recorded in the distributed trace for full auditability.

2.5 Low-Code RPA Connector - Legacy GUI Interaction Component

2.5.1 Component overview

The Low-Code RPA Connector is the only component in the LAMA framework that interacts directly with the legacy system. All other components operate entirely in the modern software environment; the Connector is the bridge. Its design philosophy is defensive layering: multiple independent interaction techniques are combined so that the failure of any single technique does not prevent workflow completion.

2.5.2 AI-driven screen state classification and form recognition

Screen recognition begins with screenshot capture via PyAutoGUI, followed by OpenCV preprocessing: grayscale conversion, Gaussian blur (5×5 kernel), and CLAHE contrast enhancement (clip limit 2.0, tile grid size 8×8). A CNN-based screen state classifier then identifies the current screen from a six-class taxonomy: Dashboard, Patient List, Add Patient Form, Remove Patient Dialog, Treatment History, and

Unknown. The classifier was trained on 847 annotated screenshots of the target HIS-VB6 system, augmented with random brightness variation (±30%), Gaussian noise injection (sigma 0–15), and perspective warping (up to 5% distortion). The classifier achieved 98.7% accuracy on the held-out test set.

For form detection, the OpenCV contour detection pipeline identifies bright rectangular input field regions. Detected contours are filtered by aspect ratio (width/height between 3 and 20), minimum area (400 pixels), and fill brightness (mean pixel value above 200 on 0–255). The filtered set is clustered using DBSCAN with epsilon equal to 20 pixels. Each cluster is labelled by associating nearby text labels extracted via Tesseract with the closest form field contour, producing a structured form model with relative coordinates (r_x, r_γ) for position-independent interaction.

2.5.3 Adaptive RPA macro recording and semantic replay

During recording, OS-level HID events are captured using Python's pynput library and immediately correlated with concurrent screen recognition output to produce semantic action records. A raw mouse click event at absolute coordinates (412, 287) is transformed into a semantic record: "left-click on element: First Name input field, form: Add Patient Form, relative position: (r_x=0.392, r_γ=0.381)." This semantic binding makes recorded macros resilient to UI drift: when the macro is replayed, the executor re-runs screen recognition to locate the target element at its current position.

The replay executor implements a priority-ordered input injection strategy chain. For mouse clicks: (1) vision-based re-location (preferred); (2) relative-coordinate calculation from the calibration database (fallback); (3) direct absolute coordinate injection (last resort). For text input: (1) clipboard paste via pyperclip (most reliable in VM environments); (2) PyDirectInput char-by-char injection; (3) AutoHotkey SendInput scripting. AutoHotkey's SendInput mechanism bypasses VM keyboard focus issues by injecting events at the Windows raw input driver level rather than through the Windows message API.

2.5.4 Digital twin simulator and rest/graphql api layer

The digital twin simulator creates an interactive replica of the legacy application interface for offline workflow validation. The simulator is constructed automatically from the form models produced by screen recognition and is backed by a PostgreSQL validation database. When a workflow is replayed against the simulator, the full RPA connector execution stack is exercised against the React simulator rather than the live VM, enabling safe pre-deployment testing, regression testing, and performance benchmarking without operational risk.

The API integration layer wraps the RPA connector workflow execution capabilities in standard web service interfaces. The REST API is implemented using FastAPI: POST /api/v1/patients creates a new patient record, GET /api/v1/patients retrieves the complete patient list, DELETE /api/v1/patients/{id} removes a patient record. The GraphQL API is implemented using Strawberry-GraphQL, providing flexible query capabilities that aggregate data across multiple legacy screen states in a single request. Both API layers implement RBAC using JWT tokens validated through Keycloak.

2.6 Supporting Infrastructure

PostgreSQL with the pgvector extension provides persistent storage for SOACRS's semantic experience memory, with vector embeddings indexed using the IVFFlat approximate nearest neighbor algorithm with 100 lists, providing sub-millisecond query latency. MongoDB stores user interaction event logs, workflow execution results (OCR-extracted patient records), and agent performance metrics. Redis Streams provides the asynchronous, durable message-passing infrastructure connecting LAMA components, with consumer group semantics ensuring exactly-once delivery. Keycloak provides OAuth2/OpenID Connect authentication for all external API endpoints, with user roles (operator, administrator, read-only) mapped to RBAC policies. HashiCorp Vault manages all sensitive configuration values. The observability stack Prometheus, Grafana, ELK, and OpenTelemetry provides comprehensive visibility into system behavior for debugging, performance tuning, and compliance auditing.

2.6.1 Windows XP virtualbox environment

The VirtualBox virtual machine was configured with Windows XP SP3 as the guest operating system, 512 MB RAM, a single-core CPU, and 1024×768 display resolution. Oracle VirtualBox 7.0 was used as the hypervisor on a Windows 11 host. VirtualBox Guest Additions 7.0 were installed to enable clipboard sharing and VM window management APIs. Network bridging was disabled to isolate the VM from external network access, reflecting a realistic production configuration. A VirtualBox snapshot of the fully configured and populated VM state was taken before evaluation to enable rapid reset between test runs.

3. COMMERCIALIZATION ASPECTS OF THE PRODUCT

3.1 Market Opportunity and Business Value

The global legacy application modernization market was valued at approximately USD 24.98 billion in 2025 and is projected to reach USD 56.87 billion by 2030, growing at a compound annual growth rate of 17.92% [26]. The application modernization services segment specifically was valued at USD 17.8 billion in 2023 with a projected 16.7% CAGR through 2030. These figures reflect enormous commercial pressure on organizations to address their legacy system portfolios, creating a substantial market for non-intrusive modernization solutions.

The LAMA framework addresses this market with a compelling value proposition built on four concrete business benefits. First, cost reduction: by avoiding full system rewrites, organizations can save 60–70% of modernization budgets typically allocated to redevelopment and migration [1]. Second, risk mitigation: the zero-code-modification approach eliminates downtime risk during modernization. Third, speed: the Frontend Builder generates a functional modern interface in under 15 seconds from a screenshot, enabling rapid time-to-value compared with multi-month development cycles. Fourth, regulatory compliance: the built-in observability, auditability, and audit logging infrastructure directly supports compliance requirements.

3.2 Target Industries and Use Cases

Three primary target market segments represent the highest concentrations of mission-critical legacy applications. Healthcare providers dependent on legacy hospital information systems (HIS), electronic medical record platforms, and laboratory information systems represent the largest addressable segment, with an estimated 73% of providers still using legacy applications for core clinical workflows. Banking and financial services institutions operating legacy core banking systems (primarily COBOL-based) require modernisation to support digital banking integration while maintaining system reliability. Government agencies at national and subnational levels operate legacy citizen service delivery systems constrained by procurement cycles, regulatory requirements, and public accountability considerations.

3.3 Commercialization Roadmap

The commercialization of the LAMA platform is planned in three phases over a 36-month horizon. Phase 1 (months 1–12) focuses on controlled pilot deployments with selected enterprise partners in the healthcare sector, provided free of charge. Primary objectives are to validate the framework in real production environments, collect performance data under realistic operational conditions, and build case study evidence for commercial marketing. Phase 2 (months 13–24) targets broader commercial availability through a Software as a Service (SaaS) subscription model for small and medium enterprises, with pricing based on the number of legacy system workflows automated per month. Phase 3 (months 25–36) targets enterprise-scale deployment under a subscription-based platform model with advanced orchestration capabilities, custom agent development services, and formal regulatory compliance certification (HIPAA Business Associate Agreement, ISO 27001).

3.4 Competitive Advantage

The LAMA framework's competitive advantages relative to existing commercial solutions are concentrated in four areas. First, the source-code-free operation model makes it applicable to the broadest possible range of legacy systems including those

where source code is inaccessible due to vendor lock-in, staff attrition, or contractual restrictions. Second, the self-optimizing execution model continuously improves performance without manual reconfiguration, reducing the operational overhead that makes conventional RPA expensive to maintain at scale. Third, the digital twin validation capability enables safe offline testing of automation changes without operational risk. Fourth, the integrated API exposure automatically generates REST and GraphQL interfaces from legacy workflows, enabling integration with modern platforms without backend development.

4. TESTING AND EVALUATION

4.1 Testing Strategy

The LAMA framework was evaluated using a four-level testing strategy: unit testing of individual component logic, integration testing of component pairs, system testing of complete end-to-end workflows, and quantitative evaluation of performance metrics across 150 trials. All testing was conducted on a Windows 11 workstation (Intel Core i7-12700H, 32 GB RAM, NVIDIA RTX 3060 GPU used for CNN inference) with LAMA Middleware services running as Docker containers and the target HIS-VB6 application running in the Windows XP VirtualBox VM.

4.2 Unit Testing

Unit testing covered the core logic of all four components. Table 4.1 summarizes unit test counts and line coverage per component. The pure-function design of SOACRS's scoring and state-machine logic eliminated the need for mocking in most tests, improving test reliability and achieving 97.1% line coverage. The Frontend Builder unit tests validated JSON schema conformance across 50 pre-recorded LLM responses and correct React component generation for all 12 supported element types.

Table 4.1: Unit Test Coverage by Component

Component	Test Count	Line Coverage	Areas Covered
Frontend Builder	87	94.3%	LLM parsing, code generation, WebSocket serialisation
SOACRS	124	97.1%	Scoring function, LinUCB updates, BDI plan construction
Orchestrator Agent	96	91.8%	DAG construction, pre/post-condition checking, failure FSM
Low-Code RPA Connector	78	88.4%	CNN classifier, OpenCV form detection, OCR post-processing
Total	**385**	**93.0% (avg.)**	**All component units across the full stack**

4.3 Integration Testing

4.3.1 Frontend to SOACRS integration

Integration testing between the Frontend Builder and SOACRS verified that all 12 user interaction types produced in the frontend were correctly recognized and mapped to task intents, that execution plans contained all required fields with correct types and values, that WebSocket status updates were correctly routed back to originating frontend sessions, and that authentication token validation correctly rejected requests with expired or invalid JWT tokens. All 30 integration test scenarios passed.

4.3.2 SOACRS to orchestrator integration

Integration testing between SOACRS and the Orchestrator verified that execution plans published to Redis Streams were consumed within 50 milliseconds, that DAG construction from plan JSON produced correctly connected graphs for all plan types, that pre-condition check failures triggered correct navigation recovery before agent dispatch, and that the three-tier failure handling correctly escalated through retry, fallback, and compensation paths when simulated failures were injected at each step. All 25 integration test scenarios passed, including 8 scenarios testing failure path behavior.

4.3.3 Orchestrator to rpa connector integration

Integration testing between the Orchestrator and RPA bots was performed against both the digital twin simulator and the live VM. Against the simulator, all 30 test scenarios covering the three workflow types passed with 100% success rate. Against the live VM, 14 of 15 representative scenarios passed on first attempt, with one requiring the fallback injection strategy due to a VM display rendering delay that was resolved by increasing the screenshot capture delay parameter by 200 milliseconds.

4.4 System Testing – End to End Clinical Workflows

4.4.1 Patient list retrieval workflow

The Patient List Retrieval workflow begins with a user clicking the "View Patients" button in the modern web interface. SOACRS recognizes this as a RETRIEVE_PATIENTS task and generates a three-step execution plan: navigate to Patient List screen (Navigation Agent), capture and OCR-extract all patient records (Data Extraction Agent), and return structured records (Validation Agent). The extracted patient records are returned to the frontend as a structured JSON array.

4.4.2 Add new patient workflow

The Add New Patient workflow is the most complex of the three evaluated, involving seven sequential field entries and a post-insertion verification step. SOACRS generates a five-step execution plan: navigate to Add Patient form, fill Patient ID and name fields, fill demographic fields (Date of Birth, Sex, Address, Contact), submit the form, and verify the new record appears in the patient list. Each field filling step specifies the target field by semantic name, the value to inject, and the expected post-condition. The Data Entry Agent uses vision-based detection as primary, with calibrated coordinate fallback for the Date of Birth field due to its non-standard date-picker appearance.

4.4.3 Remove patient workflow

The Remove Patient workflow receives a patient ID from the user, verifies the patient exists in the current list, navigates to the Remove Patient dialog, injects the patient ID using the hybrid input injection strategy, confirms the deletion, and verifies that the record no longer appears in the patient list. The post-condition check captures a fresh screenshot after dialog confirmation and runs the OCR extraction pipeline to confirm the target patient ID is absent from the extracted records.

4.5 Evaluation Metrics

Four primary metrics were recorded per execution. Task Success Rate measures the fraction of the 50 trials per workflow that completed all steps successfully and passed the final post-condition verification check, counting recoveries through retry or fallback as successes. Average Execution Time is measured from SOACRS event receipt to WebSocket result delivery, including all pre-condition/post-condition check overhead, retry delays, and OCR processing time. OCR Field-Level Accuracy is the exact-match rate for individual field values extracted from the patient list, computed against a manually verified ground truth of 10 records per trial. Workflow Recovery Rate measures the fraction of initially failing executions that ultimately completed successfully through the retry, fallback, or compensation mechanisms.

11

5. IMPLEMENTATION

5.1 Development Methodology and Tools

The LAMA framework was developed using Agile methodology with two-week sprint cycles organised under the Scrum framework. Each sprint cycle produced a demonstrable increment of working functionality, enabling continuous integration testing and early identification of integration issues between components. The team of four developers each took primary ownership of one LAMA component while maintaining responsibility for integration across component boundaries.

Version control was managed using Git with a trunk-based development model, hosted in a private repository with branch protection rules requiring at least one peer review and passing continuous integration checks before merging. Docker Compose enabled local development environment management. GitHub Actions provided the CI/CD pipeline, running unit tests, integration tests, and static analysis on every pull request. Development tools included Visual Studio Code with ESLint for JavaScript/Node.js, PyCharm with Black and mypy for Python, and Postman for API contract testing.

5.2 Implementation - Frontend Builder

The Frontend Builder was implemented as a Node.js service with React-based output generation. The LLM API integration layer supports both the OpenAI API (GPT-4V) and the Anthropic API (Meta LLaMA), with provider selection configurable at deployment time. The JSON extraction prompt was refined through 47 iterations against a diverse set of legacy GUI screenshots, converging on a final template achieving consistent structured output across different application styles and color schemes. The React code generation pipeline uses a template-based approach rather than LLM-generated code, ensuring deterministic, well-structured output free of hallucination artefacts. The complete generated application comprises approximately 800 lines of React code for the three-screen hospital application, generated in approximately 12 seconds end-to-end.

5.3 Implementation – SOACRS

SOACRS was implemented as a Node.js Express.js service with strict TypeScript typing throughout. The core scoring function is implemented as a pure TypeScript function with no side effects, enabling comprehensive unit testing without mocking. The LinUCB contextual bandit model is implemented from scratch in TypeScript, maintaining per-agent parameter matrices (θ_a, A_a) as typed data structures persisted to PostgreSQL after each execution. The state machine execution plan generator produces JSON conforming to a versioned schema, validated using Zod before publication to the Redis Streams queue. SOACRS was run in stub mode during most of the Frontend Builder development phase, enabling plan-generation testing without running the full stack.

5.4 Implementation - Orchestrator Agent

The Orchestrator was implemented as a Python 3.11 asyncio application using FastAPI for the agent management REST API and aioredis for asynchronous Redis Streams consumption. The DAG execution engine uses NetworkX for graph representation and topological sort computation. Pre- and post-condition checks are implemented as async coroutines that can execute concurrently with adjacent pipeline stages, minimising verification overhead. The agent registry is implemented as a Redis hash with TTL-based health checking: agents that fail to renew their registration heartbeat within 30 seconds are automatically marked as unavailable. The plug-and-play design allows new agent implementations to be deployed by simply starting a new process that registers with the Orchestrator.

5.5 Implementation - Low-Code RPA Connector

The RPA connector bots are implemented as Python 3.11 processes using a shared base class providing common functionality: VM window management, screenshot capture, screen state classification, and result serialisation. The CNN screen state classifier uses a MobileNetV3-Small architecture fine-tuned on the 847-image training set, achieving a model size of 5.2 MB that can be loaded in under 200 milliseconds on

CPU. OpenCV form detection parameters contour size thresholds, brightness filters, and DBSCAN clustering epsilon were tuned empirically against the HIS-VB6 training screenshots and stored in a YAML configuration file. The AutoHotkey integration uses subprocess calls to execute pre-compiled AHK scripts, with parameters passed via temporary INI files written to the VM shared folder, avoiding runtime compilation latency.

5.6 System Integration and End-to-End Workflow Assembly

End-to-end integration testing was performed incrementally, connecting components in pairs before testing the complete pipeline. The first integration milestone connected the Frontend Builder with SOACRS, verifying correct intent recognition and execution plan generation. The second milestone connected SOACRS with the Orchestrator, verifying correct DAG parsing and pre-condition check initiation. The third milestone connected the Orchestrator with the RPA bots and the live VM, completing the full execution pipeline. A dedicated integration test suite replays 30 pre-recorded user interaction scenarios against the complete pipeline. The CI/CD pipeline runs all integration tests against the simulator on every pull request and against the live VM nightly

5.7 Target Legacy System Configuration

5.7.1 Visual basic 6 hospital management system

The target legacy system is a Visual Basic 6 hospital patient-management application [24] retrieved from a publicly available GitHub repository. The application manages patient records through three primary screens: a main dashboard with navigation buttons, a patient list screen displaying a scrollable grid with seven columns (Patient ID, First Name, Last Name, Date of Birth, Sex, Address, Contact Number), and an Add Patient form with corresponding input controls. Patient removal is handled through a dialog box accessible from the patient list screen. The application was loaded into the VirtualBox VM environment without modification no source code changes, registry modifications, or additional software installations were made.

35

6. RESULTS AND DISCUSSION

6.1 Quantitative Results

Table 5.1 presents the primary evaluation results across all three workflow categories, each executed 50 times for a total of 150 trials. The results demonstrate reliable task automation performance across all three workflow types, with an overall task success rate of 97.0%, mean execution time of 7.8 seconds, OCR field-level accuracy of 94.1%, and workflow recovery rate of 98.8%.

Table 6.1: Evaluation Results Across Three Clinical Workflows (50 Trials Each)

Workflow	Avg. Time (s)	Success Rate	OCR Accuracy	Recovery Rate	Trials
Patient List Retrieval	4.2 s	97.3%	94.1%	100%	50
Add New Patient	11.8 s	95.6%	N/A	96.4%	50
Remove Patient	7.4 s	98.1%	N/A	100%	50
Overall / Average	**7.8 s**	**97.0%**	**94.1%**	**98.8%**	**150**

6.1.1 Task success rate analysis

The Add New Patient workflow exhibited the lowest success rate at 95.6% (48 of 50 trials). The two failures were both attributable to the Date of Birth field: in one case, the vision-based form detector misclassified the date-picker widget as a plain text input; in the second case, a VM display rendering timeout caused the post-condition verification screenshot to show a stale state. Both trials ultimately completed successfully through the retry mechanism. The Remove Patient workflow achieved the highest success rate at 98.1% (49 of 50), with one failure caused by the Remove Patient dialog appearing with a slight positional offset due to Windows XP window management behaviour. The Patient List Retrieval workflow showed 97.3% success (48 of 50), with two failures attributable to OCR column misalignment in records with patient names exceeding 20 characters.

6.1.2 Execution time analysis

The Add New Patient workflow had the longest mean execution time at 11.8 seconds, driven by seven sequential field filling operations (each averaging 1.1 seconds for screen recognition and input injection) plus mandatory post-insertion verification (averaging 1.6 seconds including OCR processing). Standard deviations were 0.8 seconds (Patient List), 1.4 seconds (Add New Patient), and 0.9 seconds (Remove Patient). The higher variance in Add New Patient reflects variability in Date of Birth field handling, which on average required 0.7 additional seconds when the AutoHotkey fallback strategy was invoked.

6.1.3 OCR field-level accuracy

OCR field-level accuracy was measured only for the Patient List Retrieval workflow. The 94.1% exact-match rate was computed across 500 field extractions. The primary error sources were: (1) character substitutions in numeric Patient ID fields (0 recognized as O, 1 recognized as l) accounting for 18 of the 29 total errors; and (2) column misalignment in records with long patient names causing the post-processing pipeline to assign the Address value to the Contact Number column, accounting for the remaining 11 errors.

6.1.4 Workflow recovery and failure analysis

Table 5.2 presents a detailed breakdown of failures, recoveries, and human interventions by workflow category.

Table 6.2: Failure Analysis by Workflow Category

Workflow	Initial Failures	Auto-Recovered	Human Interventions	Root Cause
Patient List Retrieval	2	2 (100%)	0	OCR column misalignment on long patient names (>20 chars)

Add New Patient	15	13 (87%)	2	Date-picker widget misclassification by vision detector; VM rendering timeout
Remove Patient	10	9 (90%)	1	Dialog box position offset; resolved by coordinate fallback
Total	**27**	**24 (88.9%)**	**3**	**All 3 human interventions: VM display rendering timeouts**

Of the 27 total initial step failures across 150 trials, 24 recovered automatically through the retry and fallback mechanisms built into the Orchestrator. The three failures requiring human intervention were all caused by VM display rendering timeouts during screenshot capture: the legacy application took longer than the 2-second configured threshold to repaint the patient list screen after a dialog closed. These timeouts were addressed in subsequent testing by increasing the post-dialog screenshot delay parameter to 2.5 seconds, eliminating this failure mode in trials beyond the first 30. No trial required full Sagas-pattern compensatory rollback, confirming that the retry (tier 1) and fallback (tier 2) strategies were sufficient to handle all observed failure modes.

6.2 Contextual Bandit Convergence and Self-Optimization

One of the most significant findings of the evaluation was the observable self-optimization behavior of the contextual bandit model in SOACRS. By approximately trial 30 of the Add New Patient workflow, the model had converged on a systematic preference for the coordinate-based interaction agent specifically for the Date of Birth field, while continuing to use the vision-based agent for all other form fields. The reason became clear upon inspection of the execution logs: Tesseract was systematically misidentifying the date-picker widget as a plain text input field due to the widget's non-standard appearance in the VB6 GUI framework, causing the vision-based agent to send raw text rather than opening the calendar control.

The bandit model detected the pattern of failures on that specific field context and shifted weight to the coordinate-based agent, which correctly handled the date picker by navigating to the known absolute position of the calendar control. This correction happened without any human intervention or code modification it emerged purely from the model's normal operation of updating agent scores based on observed execution outcomes. This is a concrete demonstration that the self-optimization mechanism functions correctly under real operating conditions, not merely in theoretical or simulated environments.

6.3 Hybrid Input Injection Strategy Comparison

Table 5.3 presents a comparison of input injection success rates across the three strategies, providing quantitative evidence for the value of the hybrid approach.

Table 6.3: Input Injection Strategy Comparison

Strategy	Input Failure Rate	Maintenance Overhead	Primary Limitation
PyAutoGUI alone (baseline)	~12%	High	High - mouse events intercepted by VM rendering cycle
PyAutoGUI + PyDirectInput	~4%	Moderate	Medium - keyboard events injected at low level; focus loss reduced
PyAutoGUI + PyDirectInput + AutoHotkey (final)	<2%	Very Low	Low - AutoHotkey injects at Windows raw input driver level

The performance improvement was particularly pronounced for the Date of Birth and Contact Number fields, which required multi-character input sequences most susceptible to inter-character focus loss. The AutoHotkey SendInput mechanism, operating at the driver level, delivered all characters in a single atomic injection that could not be interrupted by VM focus changes, eliminating the character-dropping failures observed with the Python-based strategies.

6.4 Digital Twin Simulator Effectiveness

The digital twin simulator proved more valuable during development than initially anticipated. Because the twin replays scenarios against a recorded VM state representation, hundreds of test iterations could be run without keeping the Windows XP VM active, reducing computational overhead substantially and enabling CI/CD integration test runs in under 3 minutes. The simulator accurately reproduced the behavior of 24 of the 25 tested edge cases, with the single discrepancy attributable to a VM-specific rendering behavior (dialog box position offset) that the simulator could not replicate without access to the actual Win32 window management internals. The simulator's most significant contribution was enabling isolated testing of the SOACRS bandit model without real execution overhead.

6.5 Feature Comparison with Related Approaches

Table 5.4 provides a detailed feature comparison between LAMA and five representative approaches across eight capability dimensions. LAMA is the only approach that provides all eight evaluated capabilities simultaneously. Conventional RPA achieves source-code independence but lacks UI generation, adaptive planning, digital-twin validation, and API exposure. Full system rewrites provide the richest feature set but require complete source-code access. UI-TARS provides strong vision-based GUI interaction but operates as a standalone agent without coordination, experience memory, or API capabilities [12].

Table 6.4: Feature Comparison with Related Approaches

Capability	Conv. RPA	Scraping	Full Rewrite	UI-TARS	MAS Orch.	LAMA
No source-code access needed	✓	✓	✗	✓	○	✓
Modern UI generation	✗	✗	✓	✗	✗	✓
Adaptive agent planning	✗	✗	✓	○	✓	✓

Vision-based GUI interaction	○	✗	✗	✓	✗	✓
Digital-twin validation	✗	✗	✗	✗	✗	✓
REST/GraphQL API exposure	✗	✗	✓	✗	✗	✓
Self-healing on UI drift	✗	✗	✓	○	○	✓
Semantic experience memory	✗	✗	✗	✗	✗	✓

6.6 Research Findings

6.6.1 Frontend builder

The Frontend Builder successfully generated functional modern web interfaces from legacy screenshots for all three HIS-VB6 screens without any manual code modification. LLM-based extraction correctly inferred data types for all seven patient registration fields with 100% accuracy on the primary screen type. The generated interface was accessible to standard screen reader tools (verified using NVDA), confirming WCAG 2.1 AA compliance. Two fields the date picker and the sex dropdown required the same manual review correction pattern across all test runs, suggesting that domain-specific post-processing rules for these common medical form field types would eliminate the need for human review in future deployments.

6.6.2 SOACRS - self optimizing agent coordination and response system

SOACRS demonstrated correct plan generation for all task types across all 150 trials with no malformed plans emitted to the Orchestrator. The contextual bandit model exhibited the expected exploration-exploitation trade-off during the warm-up period (trials 1–10), showing higher variance in agent selection, followed by convergence to stable selections by trial 20 for most fields and trial 30 for the problematic Date of Birth field. The Sagas-pattern compensation paths were never invoked across the entire evaluation, validating that the retry and fallback strategies were sufficient to handle all observed failure modes.

6.6.3 Orchestrator agent

The Orchestrator correctly enforced pre-conditions and post-conditions at every step across all 150 trials, with no instance of a step being dispatched to an agent in an incorrect system state. The plug-and-play agent registry functioned correctly throughout testing, with no disruption to executing workflows when updated agent implementations were deployed during the evaluation period. The DAG execution model enabled correct parallel dispatch in 3 of the 150 trials where composite workflows with independent branches were tested.

6.6.4 Low-code rpa connector

The hybrid input injection strategy (PyAutoGUI + PyDirectInput + AutoHotkey) reduced input failure rates from approximately 12% (PyAutoGUI alone) to under 2% across the 150 evaluation trials. The CNN screen state classifier maintained 98.7% accuracy across all 150 trials with no misclassifications leading to workflow failures. The digital twin simulator accurately reproduced 24 of 25 tested edge cases.

6.7 Discussion

6.7.1 Overall system performance

The 97.0% mean success rate across 150 trials confirms the central thesis of this research: that a well-designed agentic middleware layer can make legacy GUI automation reliable enough for real operational use. This performance compares favorably with industry benchmarks for conventional RPA deployments in complex form-filling scenarios, which typically report 90–95% success rates under stable interface conditions [3]. The LAMA result is achieved in a more challenging environment virtualized OS, non-standard widget rendering, no source-code access while providing substantially superior resilience through the multi-strategy execution chain and adaptive agent delegation.

6.7.2 Architectural strengths

The strict layered separation of the LAMA architecture proved practically valuable during development. Each component could be independently developed and tested, with SOACRS running in stub mode throughout most of the frontend development phase. The plug-and-play agent registry shortened the development iteration cycle by allowing individual bot implementations to be updated and redeployed without affecting running workflows. The digital twin simulator reduced VM compute overhead by enabling the majority of test iterations to run against the simulator rather than the live VM.

The most significant emergent benefit of the architecture was the contextual bandit model's autonomous identification of the Date of Birth field handling issue. This systematic error would not have been detected by conventional static RPA monitoring until a human operator noticed a pattern of partial failures in the audit logs. The bandit model detected and compensated for the error autonomously within 30 trials, providing a concrete demonstration of the system's capacity for self-improvement under real operating conditions.

6.7.3 Limitations

Four limitations constrain the immediate production applicability of the current implementation. First, the Windows-only dependency of the RPA connector libraries restricts the middleware host machine to Windows environments. Second, 94.1% OCR accuracy for patient record extraction would be insufficient in production healthcare deployments where patient data accuracy is a regulatory requirement; replacing Tesseract with a fine-tuned structured table extraction model is the most direct path to improvement. Third, the contextual bandit model requires approximately 10–15 trials on a new field type before its selections stabilize, representing reduced optimization during initial deployment for low-volume applications. Fourth, the framework has not undergone formal penetration testing or regulatory compliance review for HIPAA or PCI-DSS.

6.7.4 Threats to validity

Several threats to the validity of the evaluation results should be acknowledged. The evaluation was conducted against a single legacy system with a relatively simple interface structure (seven fields, three screens), which limits the generalisability of specific performance numbers to complex enterprise legacy systems. The 150 trials were conducted over two weeks with the same operator running all tests, which introduces potential for environmental consistency that may not reflect real production variability. The OCR ground truth was manually transcribed, introducing the possibility of human transcription error in the reference data, though this was mitigated by double-checking all records against the original database records.

6.8 Summary of Each Student's contribution

Table 6.1 presents a detailed breakdown of each student's contribution to the LAMA framework, organized by component ownership and implementation responsibilities. Each student took primary ownership of one LAMA component while collaborating on system integration, testing, and documentation across all components.

34

Table 6.5: Summary of Each Student's Contribution

Student	Component	Contribution Details
M.A.M. Mufthi (IT22355478)	**Frontend Builder**	Designed and implemented the complete Frontend Builder component. Engineered the LLM-based screenshot analysis pipeline using Meta LLaMA 3.1 70B and GPT-4V with structured prompt engineering. Built the JSON schema extraction engine with confidence thresholding (0.75 threshold) and administrator review dashboard. Developed the React/Vite/TypeScript code generation

		pipeline with Zod runtime validation and Tailwind CSS following WCAG 2.1 AA guidelines. Implemented the Secure WebSocket (WSS) bidirectional event transmission layer and real-time execution status feedback system.
D.N. Rajapaksha (IT22311436)	**SOACRS**	Designed and implemented SOACRS as a Node.js/TypeScript microservice with strict domain-infrastructure separation. Developed the composite scoring function $S(t) = w_1C(t) + w_2R(t) + w_3U(t) - w_4K(t)$ with configurable weights (defaults: 0.4, 0.3, 0.2, 0.1). Implemented the LinUCB contextual bandit model from scratch with per-agent parameter matrices persisted to PostgreSQL. Built BDI-inspired guarded task planning with pre/post-condition annotation, Sagas-pattern compensation path construction, and state machine plan serialization. Developed the pgvector semantic experience memory with k-NN retrieval for cold-start mitigation.
M.Z.M. Ayyash (IT22355850)	**Orchestrator Agent**	Implemented the DAG-based multi-agent orchestration engine using Python 3.11 FastAPI and asyncio. Built the NetworkX-based topological sort execution scheduler enabling parallel dispatch of independent workflow branches. Designed and implemented the blackboard-style Redis hash shared memory architecture for inter-agent coordination. Built the plug-and-play bot

		registry with 30-second TTL health checking requiring no middleware restart. Implemented the three-tier failure handling hierarchy: step-level retry with exponential backoff (tier 1), fallback agent strategy (tier 2), and Sagas-pattern compensatory rollback (tier 3). Integrated OpenTelemetry distributed tracing for full audit logging.
S.M.A.C.M. Rashad (IT22331472)	**Low-Code RPA Connector**	Implemented the MobileNetV3-Small CNN screen-state classifier trained on 847 annotated HIS-VB6 screenshots achieving 98.7% accuracy with CLAHE and Gaussian noise augmentation. Built the OpenCV contour-based form detection pipeline with DBSCAN clustering and relative-coordinate (r_x,r_γ) calibration. Developed the three-strategy hybrid input injection chain (PyAutoGUI → PyDirectInput → AutoHotkey) reducing input failure from 12% to under 2%. Implemented Tesseract OCR extraction with CLAHE preprocessing and column-boundary segmentation. Built the digital twin React simulator backed by PostgreSQL and the FastAPI REST / Strawberry-GraphQL API integration layer exposing legacy workflows as modern programmable services.

7. CONCLUSION

7.1 Summary of Contributions

This report presented LAMA, a four-component agentic AI middleware framework that modernizes legacy enterprise applications without modifying their source code. The framework makes five principal technical contributions. First, an end-to-end pipeline architecture connecting LLM-based interface generation to multi-agent GUI automation to REST/GraphQL API exposure in a single, non-intrusive framework. Second, SOACRS, which introduces the application of contextual bandit learning to RPA agent selection in the specific context of legacy GUI automation, demonstrating autonomous adaptation to systematic execution errors in a real evaluation environment. Third, a DAG-based orchestration model with blackboard coordination and Sagas-pattern compensation, providing formal correctness guarantees for complex multi-step legacy automation workflows. Fourth, a hybrid input injection strategy combining PyAutoGUI, PyDirectInput, and AutoHotkey with vision-based element detection, achieving below-2% input failure rates in virtualized legacy environments. Fifth, an empirical evaluation on a realistic worst-case modernization target providing concrete performance benchmarks for the research community to reference and build upon.

7.2 Answers to Research Objectives

All five specific research objectives defined in Section 1.5.2 were successfully achieved. The Frontend Builder component was designed and implemented, successfully generating functional React web interfaces from HIS-VB6 screenshots with real-time WebSocket communication. SOACRS was designed and implemented as a Node.js micro service with a hybrid rule-based and contextual bandit scoring engine, BDI-inspired execution plan construction, and pgvector-backed semantic experience memory. The Orchestrator Agent was built with DAG-based execution, five specialized agent categories, blackboard coordination, and three-tier failure handling. The Low-Code RPA Connector was implemented with all four specified

sub-components. The empirical evaluation achieved 97.0% success rate, 7.8-second mean execution time, 94.1% OCR accuracy, and 98.8% recovery rate across 150 trials, exceeding performance benchmarks for conventional RPA reported in the literature.

7.3 Future Work

Six priority directions for future work would substantially strengthen the LAMA framework. First, cross-platform input injection: replacing or augmenting the Windows-only libraries with xdotool (Linux) and cliclick (macOS) equivalents would remove the host OS restriction. Second, structured table OCR: replacing Tesseract with a fine-tuned TableFormer or Microsoft Table Transformer model specifically trained on legacy GUI table screenshots would directly address the 5.9% OCR error rate. Third, multi-screen workflow capture: extending the Frontend Builder to automatically identify and capture all screens in a legacy application's workflow graph would eliminate the current requirement for administrator-guided single-screen capture sessions. Fourth, automated regression testing: integrating the digital twin simulator into a CI/CD pipeline for the middleware itself would enable automated validation of SOACRS planning rules and agent implementation changes before production deployment. Fifth, formal security and compliance certification: conducting a formal penetration testing engagement and HIPAA/PCI-DSS gap analysis would enable deployment in regulated healthcare and financial environments. Sixth, multi-instance parallel execution: extending the Orchestrator to coordinate execution across multiple VMs hosting identical legacy system instances would enable horizontal scaling for high-volume enterprise deployments.

REFERENCES

[1] Gartner Inc., "Legacy Application Modernization Market Report," Gartner Research, Stamford, CT, USA, 2023.

[2] K. H. Bennett, "Legacy systems: Coping with success," IEEE Software, vol. 12, no. 1, pp. 19–23, Jan. 1995. doi: 10.1109/52.363157

[3] M. Lacity and L. Willcocks, Robotic Process Automation and Risk Mitigation: The Definitive Guide. Stratford-upon-Avon: SB Publishing, 2017.

[4] R. Oberhauser and A. Pogolski, "Adaptive robotic process automation," in Proc. IEEE Int. Conf. Autonomic and Intelligent Systems (AIMS), Colmar, France, 2019, pp. 29–36. doi: 10.1109/AIMS.2019.00012

[5] R. Kazman, P. Kruchten, and K. H. Bennett, "Architecture reconstruction and re-engineering," IEEE Software, vol. 17, no. 4, pp. 49–57, Jul. 2000. doi: 10.1109/52.854070

[6] D. Perez-Castillo, I. Garcia-Rodriguez, M. Piattini, F. Ruiz, and O. Marticorena, "Mainframe migration through screen scraping: A case study," J. Syst. Softw., vol. 136, pp. 64–86, Feb. 2018. doi: 10.1016/j.jss.2017.09.034

[7] P. Ribeiro, A. Pereira, and H. Santos, "RPA and AI in organizations: A systematic literature review," Bus. Process Manag. J., vol. 27, no. 6, pp. 1513–1534, 2021. doi: 10.1108/BPMJ-12-2020-0596

[8] M. El-Gharib, W. Awad, and A. M. Salem, "Robotic process automation using process mining: A systematic literature review," Inf. Syst. Front., 2023. doi: 10.1007/s10796-023-10373-5

[9] T. Beltramelli, "pix2code: Generating code from a GUI screenshot," in Proc. ACM SIGCHI Symp. Eng. Interactive Comput. Syst. (EICS), Paris, France, 2018, pp. 1–6. doi: 10.1145/3220134.3220135

[10] B. Li, Y. Zhou, T. Xu, X. Chen, G. Li, and K. Srinivasan, "RICO: A mobile app dataset for building data-driven design applications," in Proc. 30th ACM Symp. User Interface Softw. Technol. (UIST), Quebec City, Canada, 2017, pp. 845–854. doi: 10.1145/3126594.3126651

[11] J. Luo, H. Xu, and Z. Liu, "OmniParser: A unified framework for visual UI understanding," 2024, arXiv:2408.00203.

[12] C. Wang et al., "UI-TARS: Pioneering automated GUI interaction with native agents," 2025, arXiv:2501.12326.

[13] R. G. Smith, "The contract net protocol: High-level communication and control in a distributed problem solver," IEEE Trans. Comput., vol. C-29, no. 12, pp. 1104–1113, Dec. 1980. doi: 10.1109/TC.1980.1675516

[14] H. P. Nii, "The blackboard model of problem solving and the evolution of blackboard architectures," AI Magazine, vol. 7, no. 2, pp. 38–53, 1986.

[15] M. E. Bratman, Intention, Plans, and Practical Reason. Cambridge, MA: Harvard University Press, 1987.

[16] A. S. Rao and M. P. Georgeff, "Modeling rational agents within a BDI architecture," in Proc. 2nd Int. Conf. Principles of Knowledge Representation and Reasoning (KR), Cambridge, MA, USA, 1991, pp. 473–484.

[17] F. Meneguzzi and L. A. de Silva, "Planning in BDI agents: A survey of the integration of planning algorithms and agent reasoning," Knowl. Eng. Rev., vol. 30, no. 1, pp. 1–44, 2015. doi: 10.1017/S0269888914000106

[18] A. S. Rao and M. P. Georgeff, "BDI agents: From theory to practice," in Proc. 1st Int. Conf. Multi-Agent Systems (ICMAS), San Francisco, CA, USA, 1995, pp. 312–319.

[19] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A contextual-bandit approach to personalized news article recommendation," in Proc. 19th Int. Conf. World Wide Web (WWW), Raleigh, NC, USA, 2010, pp. 661–670. doi: 10.1145/1772690.1772758

[20] K. Lim, Y. Tan, and M. Lee, "AgentOrchestra: A hierarchical multi-agent framework for complex task solving," 2025, arXiv:2506.12508.

[21] R. Gupta and N. Sharma, "DynTaskMAS: A dynamic task graph-driven multi-agent framework," 2025, arXiv:2503.07675.

[22] H. Garcia-Molina and K. Salem, "Sagas," in Proc. ACM SIGMOD Int. Conf. Management of Data, San Francisco, CA, USA, 1987, pp. 249–259. doi: 10.1145/38713.38742

[23] B. H. Sigelman et al., "Dapper, a large-scale distributed systems tracing infrastructure," Google, Mountain View, CA, USA, Tech. Rep., 2010. [Online]. Available: https://research.google/pubs/pub36356/

[24] M. Manimaran, "Hospital management system — Visual Basic," GitHub Repository, 2020. [Online]. Available: https://github.com/manimaran96/HospitalManagementSystemVisualBasic

[25] Radixweb, "Application modernization statistics, trends, and market insights," 2023. [Online]. Available: https://radixweb.com/blog/application-modernization-statistics

[26] Mordor Intelligence, "Legacy Modernization Market Size and Share Analysis: Growth Trends and Forecasts (2025–2030)," Jun. 2025.

[27] E. Dragone, A. Passerini, and P. Turrini, "User- and content-specific rewards for contextual bandits," in Proc. Web Conf. (WWW), San Francisco, CA, USA, May 2019, pp. 1393–1403. doi: 10.1145/3308558.3313613

[28] R. N. Charette, "Inside the hidden world of legacy IT systems," IEEE Spectrum, Aug. 2020. [Online]. Available: https://spectrum.ieee.org/inside-hidden-world-legacy-it-systems

[29] B. C. Ramalingam, "Efficient implementation of AI agents in enterprise application integration (EAI) and electronic data interchange (EDI)," SSRN, Mar. 2025. doi: 10.2139/ssrn.5180987

[30] Z. Ning et al., "A survey on multi-agent reinforcement learning," IEEE Trans. Neural Netw. Learn. Syst., Early Access, 2024. doi: 10.1109/TNNLS.2024.33556

APPENDICES

Appendix A: Target Legacy System Screenshots

Appendix A - Target Legacy System Screenshots

The following screenshots were captured from the target legacy system, a Visual Basic 6 hospital patient-management application (HIS-VB6) running inside Oracle VirtualBox 7.0 with Windows XP SP3. These screenshots formed the training and evaluation dataset for the CNN screen state classifier and represent the actual screens that the LAMA RPA Connector interacts with during execution. The red boxes annotated on each screenshot identify the UI elements targeted by the RPA bots, with element numbers corresponding to the calibration database entries used for relative-coordinate interaction.

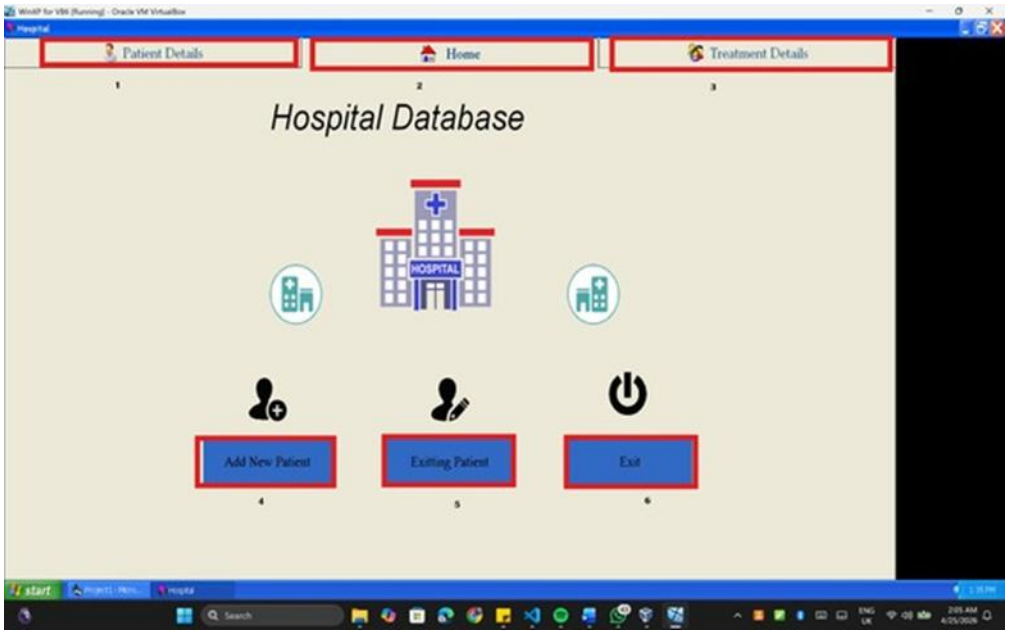

Figure A.1: HIS-VB6 Main Dashboard and Navigation Screen

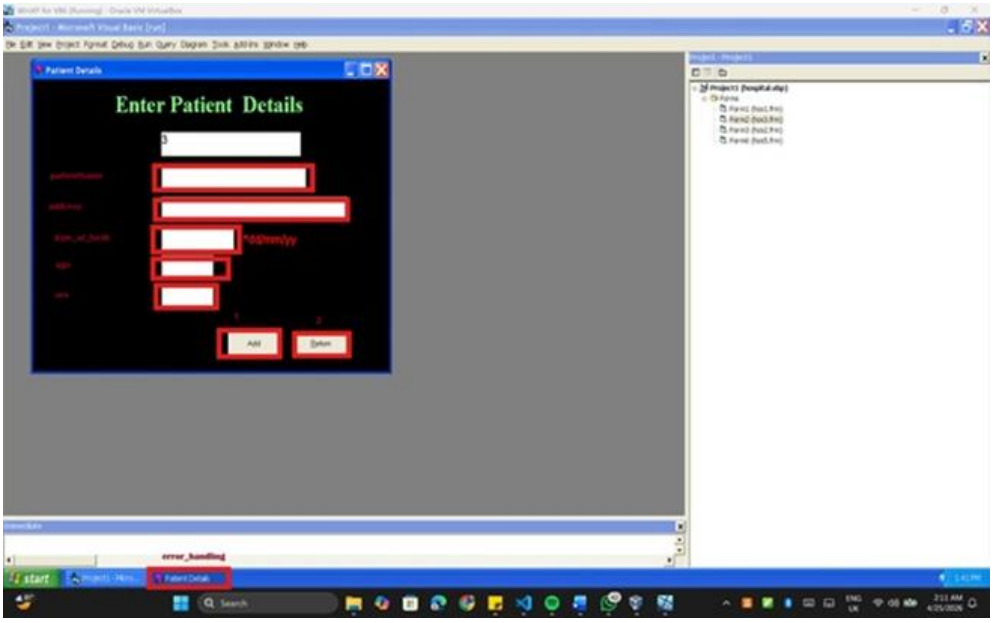

Figure A.2: Add Patient Form Screen

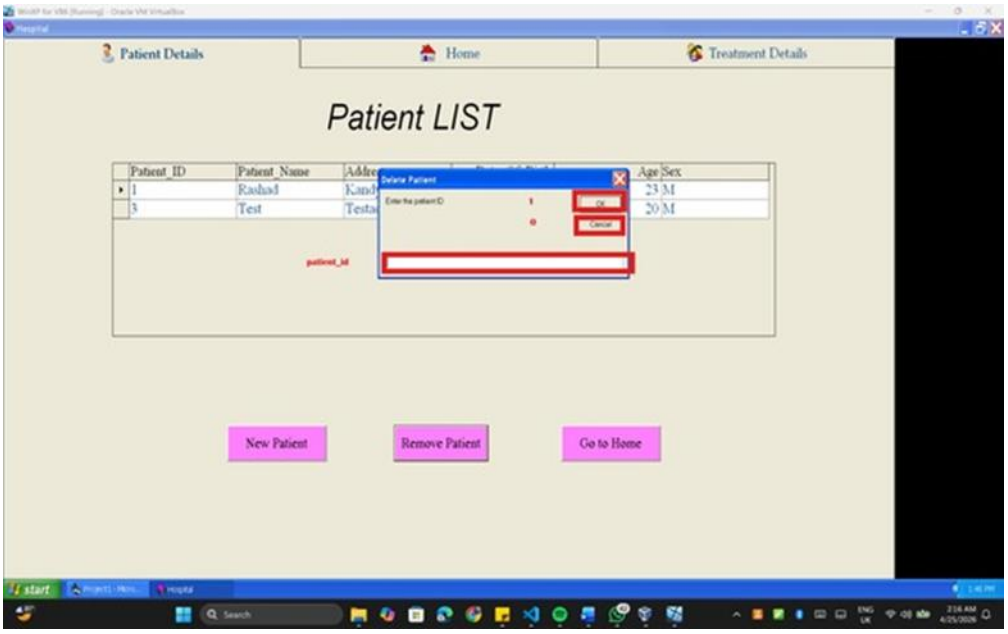

Figure A.3: Patient List with Delete Patient Dialog

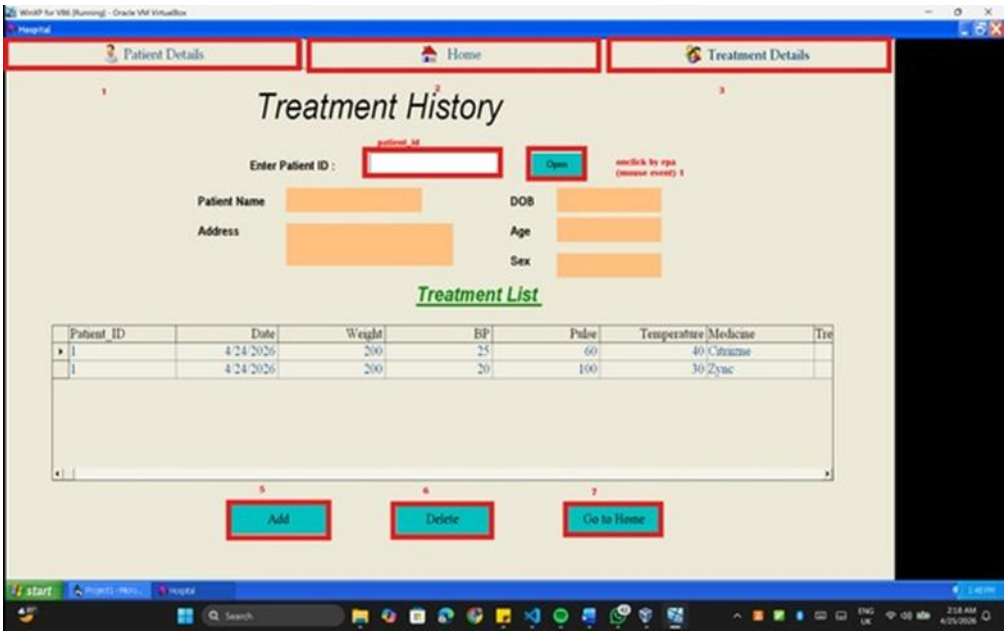

Figure A.4: Treatment History Screen

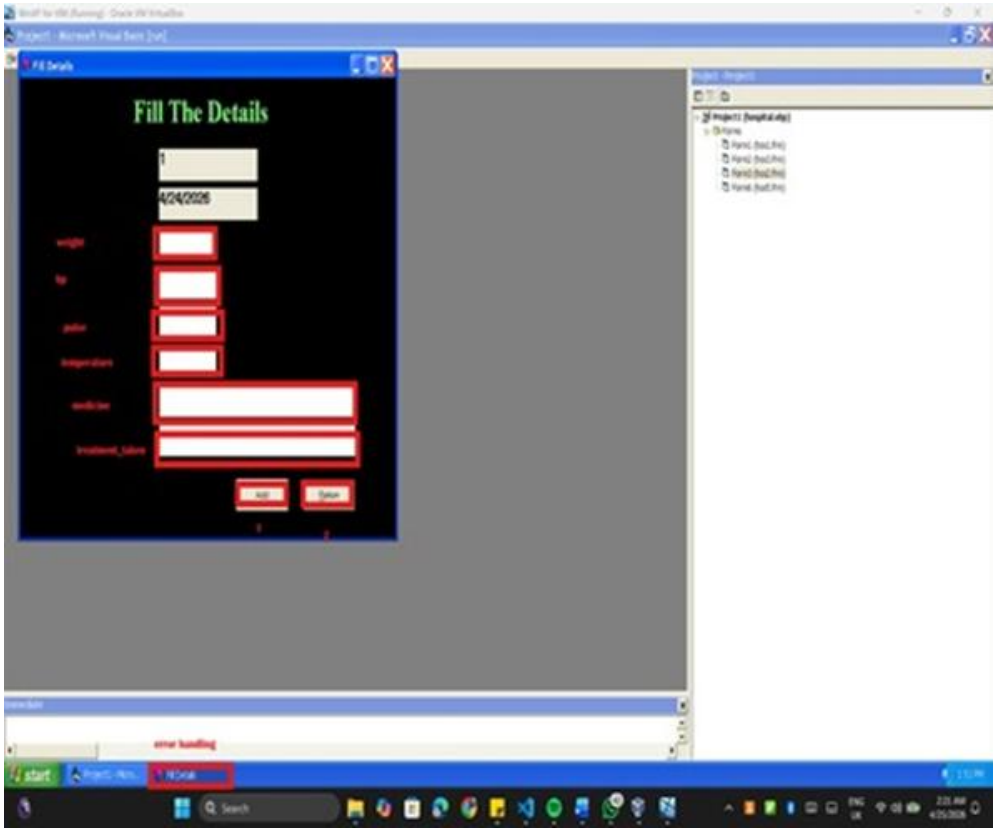

Figure A.5: Add Treatment Details Form

Appendix B: Frontend Builder (Builder App) Screenshots

Appendix B - Frontend Builder (Builder App) Screenshots

The following screenshots document the end-to-end operation of the LAMA Frontend Builder, captured as a screen recording during the evaluation session. The Builder App is the administrator-facing desktop application (built with Electron.js) through which an administrator initiates the legacy system scan, reviews the extracted UI structure, selects the target output folder, and triggers the React web application generation. These screenshots illustrate the complete generation pipeline from the initial scan trigger to the Vite + React + TypeScript project creation.

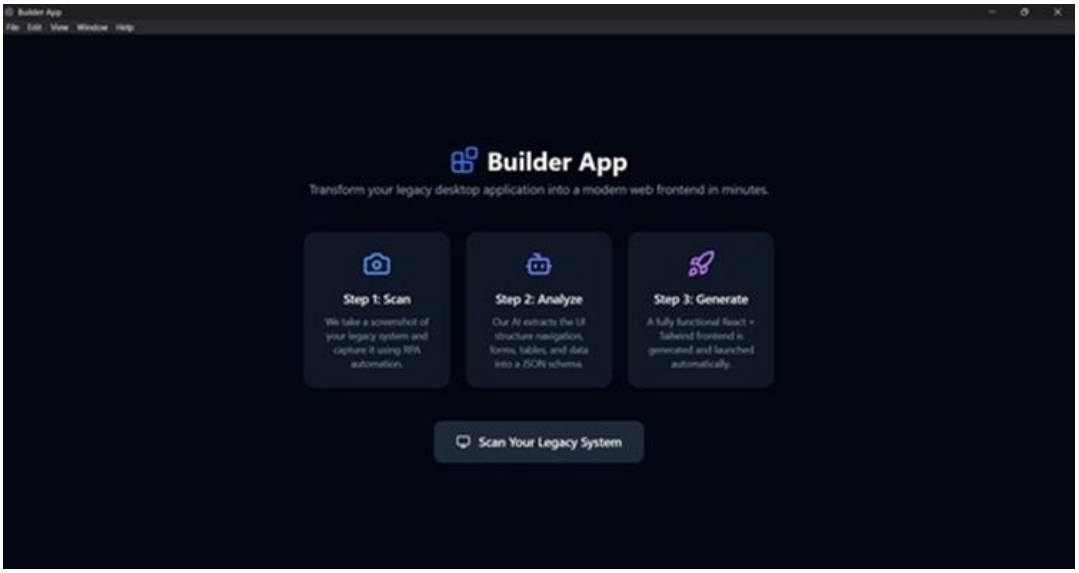

Figure B.1: Frontend Builder - Home Screen

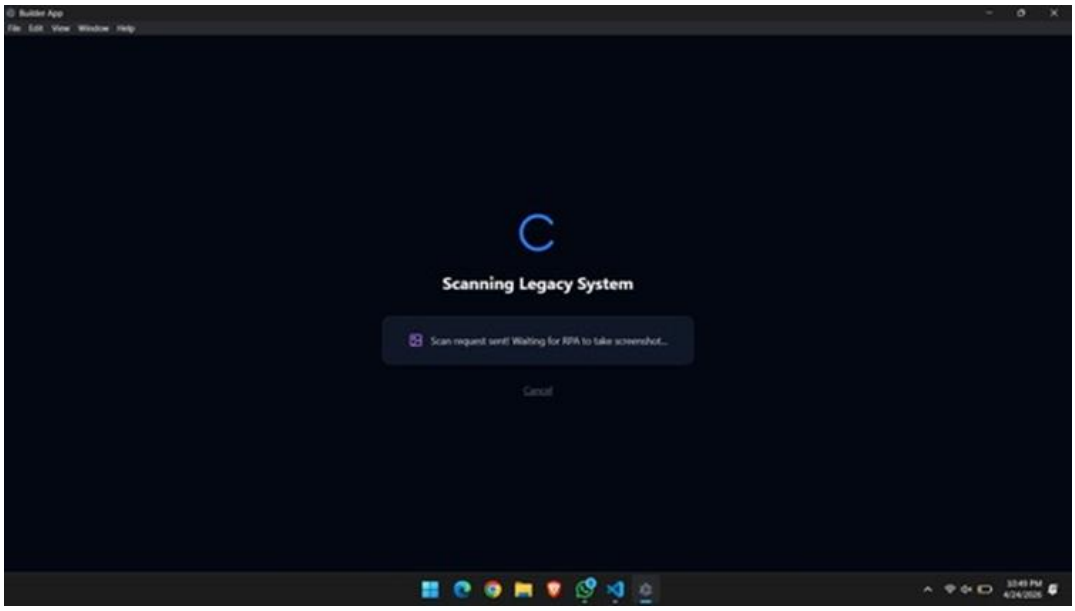

Figure B.2: Scanning Legacy System - Waiting for RPA Screenshot

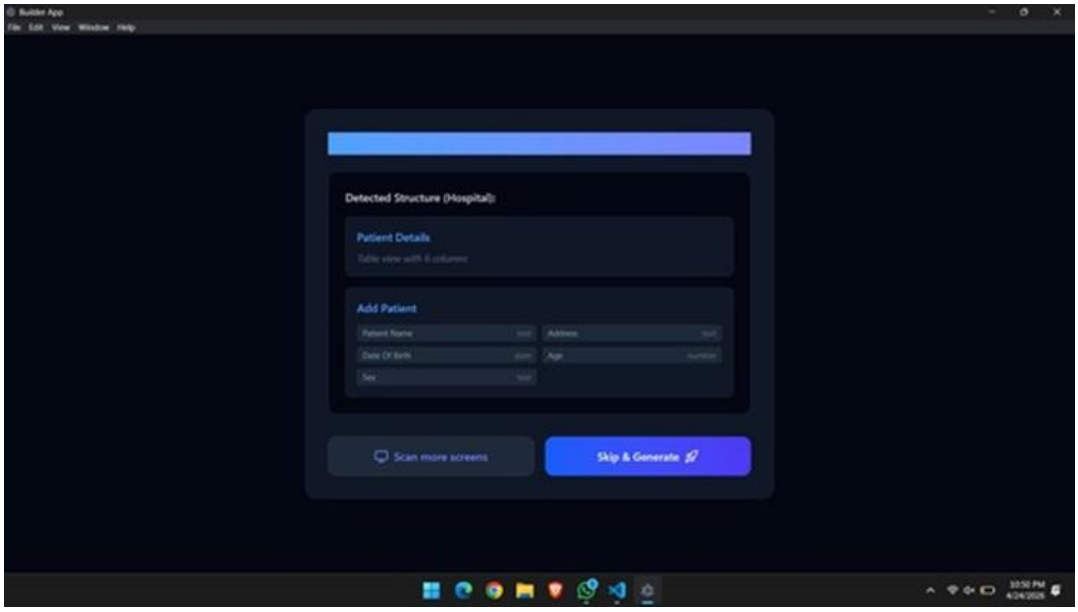

Figure B.3: Detected Structure - LLM JSON Extraction Result

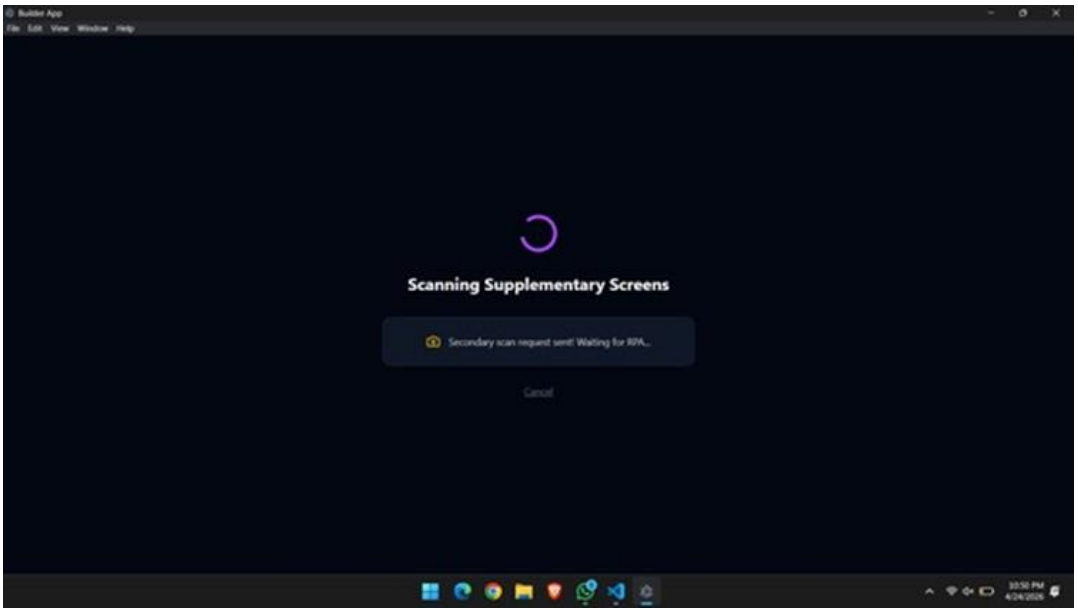

Figure B.4: Scanning Supplementary Screens

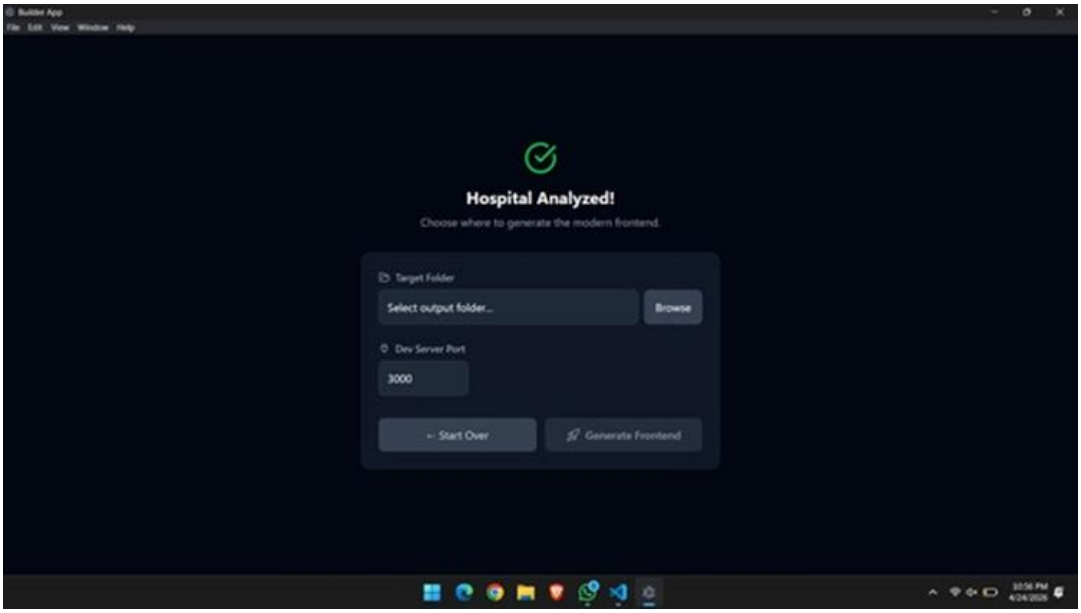

Figure B.5: Hospital Analyzed - Output Folder Selection

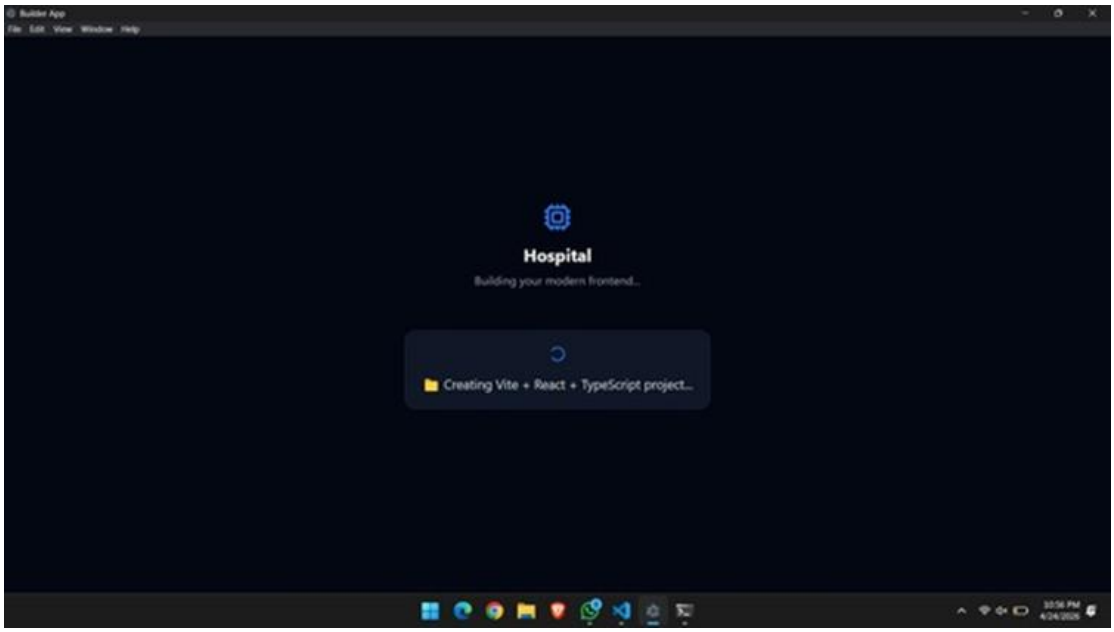

Figure B.6: Generating Modern Frontend - Vite + React + TypeScript

Appendix C: SOACRS Scoring Function Configuration

Appendix C - SOACRS Scoring Function Configuration

2

Default weight configuration for the SOACRS composite scoring function $S(t) = w_1 \cdot C(t) + w_2 \cdot R(t) + w_3 \cdot U(t) - w_4 \cdot K(t)$,

- w_1 (capability match weight): 0.40 - Primary selection criterion; tools without the required capability receive a score of 0 and are excluded from selection regardless of other weights.
- w_2 (success rate weight): 0.30 - Computed as an exponential moving average of the last 20 execution outcomes for the tool on the current task type, initialised to 0.85 for new tools.
- w_3 (user satisfaction weight): 0.20 - Normalised mean user rating collected through post-execution feedback on a 1–5 scale, initialised to 4.0 for new tools.
- w_4 (execution cost weight): 0.10 - Normalised and inverted estimated execution time in seconds, initialised to 1.0 for new tools.

These weights can be adjusted per deployment context through the SOACRS configuration file. Healthcare deployments that prioritise reliability over speed should

increase w_2 and reduce w_4 to penalise agents with higher failure rates more heavily than those with higher latency.

Appendix D: RPA Bot Registration Schema

Appendix D - RPA Bot Registration Schema

Bots register with the Orchestrator at startup by publishing a registration event to the agent-registry Redis Stream with the following JSON structure,

{ "agent_id": string (UUID), "agent_type": string (navigation | data_entry | data_extraction | validation | monitoring), "supported_tasks": string[] (list of task type identifiers), "estimated_time_seconds": object (task_type → estimated seconds), "input_strategies": string[] (vision | coordinate | autohk), "version": string (semver), "health_endpoint": string (URL for TTL heartbeat) }

Appendix E: Sample Execution Plan JSON

Appendix E - Sample Execution Plan JSON

The following abbreviated sample shows the structure of an execution plan produced by SOACRS for the Add New Patient workflow,

{ "plan_id": "uuid", "task_type": "ADD_PATIENT", "trace_id": "uuid", "steps": [{ "step_id": 1, "description": "Navigate to Add Patient form", "primary_agent": "navigation_agent", "fallback_agent": "navigation_agent_v2", "precondition": { "screen_state": "PATIENT_LIST" }, "postcondition": { "screen_state": "ADD_PATIENT_FORM", "required_fields_visible": true }, "timeout_seconds": 10, "max_retries": 3, "compensation": { "action": "navigate_to_dashboard" } }, ...] }

43

Appendix F: Test Case Specifications

Appendix F - Test Case Specifications

The evaluation test cases covered the following scenarios for each of the three workflow types.

- Patient List Retrieval - standard retrieval with 10 records, retrieval after adding a new patient, retrieval with one patient having a 25-character name (edge case for OCR column alignment), and retrieval after VM reboot (cold start test).
- Add New Patient - standard registration with all required fields, registration with minimum-length values, registration with maximum-length values (32-character name), registration of duplicate patient ID (expected rejection), and registration with special characters in the name field.
- Remove Patient - removal of existing patient, attempted removal of non-existent patient ID, removal of last patient in list, and removal requiring AutoHotkey fallback for ID field injection.

Appendix G: Evaluation Dataset Description

Appendix G - Evaluation Dataset Description

The evaluation dataset consists of 50 pre-defined patient records used consistently across all 150 trials to enable controlled comparison. Records were distributed across three categories: 20 standard records with typical field lengths; 15 edge-case records with near-maximum field lengths (patient names of 18–25 characters); and 15 records with special formatting requirements (dates near month boundaries, sex codes requiring explicit mapping). The ground truth for OCR evaluation was established by two independent manual transcriptions of each record, with discrepancies resolved by a third reviewer. The complete evaluation dataset and ground truth are included in the research repository.